



TEMA 3_1 – PROCESOS Y TAREAS LINUX

Procesos - Tareas

1. Los procesos.....	2
1.1. Concepto de proceso.....	2
1.2. Creación de procesos.....	3
1.3. Ejecución de procesos.....	3
1.4. Terminación de procesos.....	4
1.5. Tipos de procesos.....	5
1.6. Estados de un proceso.....	6
1.7. Implementación de los procesos.....	6
1.8. Concepto de Hilo.....	7
2. LINUX. Gestión de Procesos.....	8
2.1. Comandos para la gestión de procesos.....	11
2.2. Gestión de procesos en el entorno gráfico.....	21
2.3. El directorio virtual /proc.....	22
2.4. Permisos especiales ficheros.....	23
2.5. Otros Comandos - Rendimiento del sistema.....	24
3. LINUX. El proceso de arranque – Gestión de servicios.....	26
3.1. El proceso de arranque - etapas.....	26
3.2. Extraer información sobre el proceso de arranque.....	28
3.3. El proceso de inicialización System V.....	29
3.4. El proceso de inicialización Systemd.....	32
4. LINUX. Automatización de tareas.....	36
4.1. Ejecutar tareas a una determinada hora: at.....	36
4.2. Ejecutar tareas periódicamente: cron.....	38
4.3. Anacron.....	40

1. Los procesos

1.1. Concepto de proceso

Un **proceso** es en esencia un programa en ejecución.

Un proceso no es lo mismo que un programa. Un programa es un conjunto de instrucciones y datos almacenados en un archivo (ejecutable). Cuando lo que tiene ese programa se carga en memoria y se pone en ejecución, se convierte en proceso.

Un **proceso** es una entidad activa y dinámica que crea el sistema operativo para ejecutar un programa. Cada proceso tiene asociado un espacio de direcciones (memoria principal asignada al proceso) que contiene el programa ejecutable, los datos del programa y la pila de ejecución.

También hay asociado a cada proceso un conjunto de recursos, que comúnmente incluye registros de la CPU (el contador de programa ..), una lista de archivos abiertos, alarmas pendientes, listas de procesos relacionados y toda la demás información necesaria para ejecutar el programa. En esencia, un proceso es un recipiente que guarda toda la información necesaria para ejecutar un programa.

Un mismo programa en ejecución dos veces genera dos procesos distintos (aunque compartan la zona de memoria principal que almacena el código del programa) .

Los procesos los gestiona el sistema operativo y proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible.

Los sistemas operativos modernos son **multitarea** o **multiprogramados** y permiten tener varios o muchos procesos activos (el editor de video, el navegador Web, el antivirus).

La CPU conmuta de un proceso a otro con rapidez, dando la apariencia de un pseudoparalelismo (porque en sentido estricto, en cualquier instante una CPU está ejecutando sólo un proceso*).

El planificador de procesos, mediante un algoritmo de planificación, será el que determine cuándo se debe detener un proceso para ejecutar otro. Cuando el sistema operativo decide detener temporalmente la ejecución de un proceso debe guardar toda la información acerca del proceso, porque luego se debe reiniciar exactamente en el mismo estado que tenía cuando se detuvo.

* Nota: La restricción actual es que, en cada momento, solo puede existir un proceso en ejecución por núcleo de procesador. Los sistemas multiprocesador, multinúcleo o con multihilos permiten ejecutar varios procesos en paralelo. Con todo, utilizan el mismo

sistema de división de tempo para administrar casos en los existan más procesos activos que núcleos de procesador disponibles.

Así, en los sistemas operativos **multitarea** hay varios programas cargados en memoria principal, porque todo proceso para ejecutarse tiene que estar en memoria. Estos se ejecutan independientemente unos de los otros, es decir, un programa se ejecuta de la misma forma, haya o no más programas ejecutándose a la vez. Sin embargo, cuando un trabajo se divide en varios procesos que cooperan entre si para realizar dicho trabajo, es necesario que se comuniquen, para transmitirse datos y órdenes, y que se sincronicen en la ejecución de sus acciones.

La vida de un proceso se puede descomponer en las siguientes fases:

- **El proceso se crea.**
- **El proceso se ejecuta.**
- **El proceso termina o muere.**

1.2. Creación de procesos

Los eventos principales que provocan la creación de procesos son:

- **El arranque del sistema:** Generalmente, cuando se inicia un sistema operativo se crean varios procesos. Algunos son procesos en primer plano que interactúan con el usuario (ej: terminal, entorno gráfico, ..). Otros son procesos en segundo plano que están asociados con una función específica; actúan por detrás y la mayor parte del tiempo están a la espera (impresión, recepción de correo, ..).
- **La ejecución, desde un proceso, de una llamada al sistema para crear un proceso nuevo.**
- **Una petición del usuario** para crear un proceso: Los usuarios pueden iniciar un programa escribiendo un comando o haciendo (doble) clic en un icono.
- **Inicio de un proceso por lotes (batch).**

Técnicamente, en todos estos casos, para crear un proceso hay que hacer que un proceso existente ejecute una llamada al sistema de creación de proceso. Esta llamada al sistema indica al sistema operativo que cree un proceso y le indica, directa o indirectamente, que programa debe ejecutarlo.

En Unix/Linux solo hay una llamada al sistema para crear un proceso: **fork**.

- **fork():** un proceso (padre) clona su contexto y crea un proceso hijo casi idéntico.
- **exec():** el hijo (o el mismo proceso) reemplaza su imagen por un nuevo programa (carga binario/ELF o script).

- **clone()**: variante de baja nivel para crear hilos o procesos con espacios de recursos compartidos (base de los hilos en Linux).

Esta llamada crea un clon exacto del proceso que hizo la llamada.

Tanto en Unix/Linux como en Windows, una vez que se crea un proceso, el padre y el hijo tienen sus propios espacios de direcciones distintos. Si cualquiera de los procesos modifica una palabra en su espacio de direcciones, esta modificación no es visible para el otro proceso.

En algunos sistemas, cuando un proceso crea otro, el proceso padre y el proceso hijo continúan asociados. El proceso hijo puede crear más procesos, formando una jerarquía de procesos:

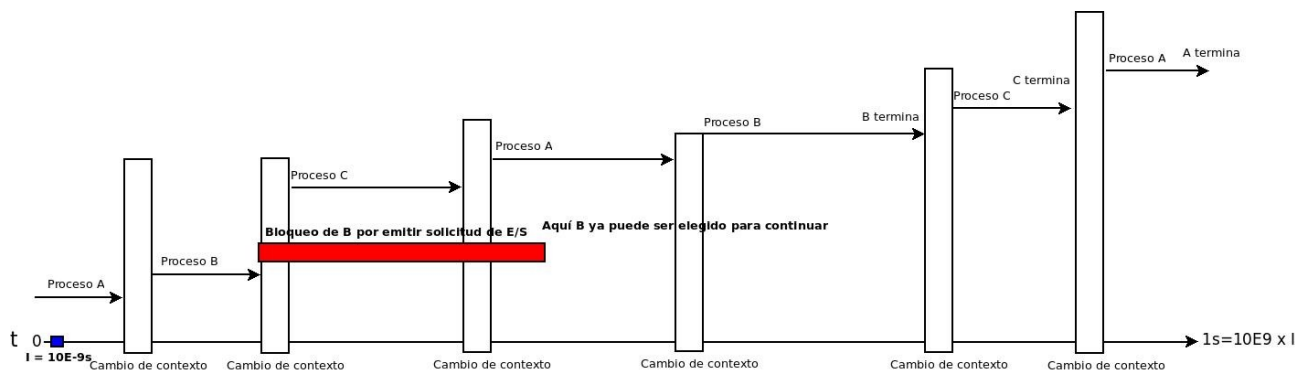
- En Unix/Linux, **un proceso y todos sus hijos, junto con sus descendientes posteriores, forman un grupo de procesos**. Los procesos en Unix/Linux no pueden desheredar a sus hijos.

1.3. Ejecución de procesos

El objetivo de un proceso es precisamente ejecutar. **Esta ejecución consiste en ráfagas de procesamiento y ráfagas de espera.**

Mientras el proceso se está ejecutando pueden ocurrir interrupciones. Bien sea porque llega una interrupción externa, una excepción o una llamada al sistema (el proceso solicita un servicio al sistema operativo), el proceso para su ejecución. Inmediatamente entra a ejecutarse el sistema operativo (cambio de modo usuario a modo privilegiado). Más tarde el sistema operativo activará este proceso, es decir continuará su ejecución en el estado exacto como lo dejó antes (vuelve a cambiar a modo usuario).

El sistema operativo esta constantemente desalojando un proceso de la CPU y asignándola a otro. A esta operación se llama **cambio de contexto**. **El cambio de contexto** requiere guardar el estado del proceso desalojado, para que pueda continuar su ejecución más tarde como si nada hubiese ocurrido. Cuando a un proceso se la asigna la CPU, se recupera todo su estado.



La CPU se "reparte" entre los procesos listos para ejecutarse: A, B, C

La parte del sistema operativo que decide qué proceso debe ejecutarse y por cuanto tiempo es el **planificador de procesos**. El planificador se encarga del manejo de las interrupciones y los detalles relacionados con iniciar y detener los procesos. La planificación de procesos es muy importante; se idearon muchos algoritmos para tratar que el sistema operativo sea lo más eficiente posible. El algoritmo que utiliza se conoce como **algoritmo de planificación**. Alguno de estos algoritmos utilizan la prioridad del proceso, que es un valor numérico que se usa como factor para determinar si un proceso debe entrar en CPU antes que otro(s).

Los procesos a menudo necesitan comunicarse entre sí, para transmitirse datos y ordenes, para sincronizar sus actividades. A esta comunicación se le conoce como **comunicación entre procesos** (IPC). Puede ocurrir que:

- Un proceso necesita pasar información a otro.
- Se necesita controlar que dos o más procesos no se interfieran entre ellos.

Por ejemplo, dos procesos en un sistema de reserva de vuelos, cada uno de ellos tratando de obtener el último asiento disponible para un cliente distinto.

- Se necesita garantizar la secuencia apropiada de los procesos cuando hay dependencias entre ellos. Por ejemplo, si un proceso A produce datos y el proceso B los imprime; B tiene que esperar hasta que A genere algunos datos antes de empezar a imprimir.

El IPC provee un mecanismo que permite a los procesos comunicarse y sincronizarse entre sí: memoria compartida, paso de mensajes, semáforos, ...

1.4. Terminación de procesos

Una vez que se crea un proceso, empieza a ejecutarse y realiza el trabajo al que está destinado. Tarde o temprano el nuevo proceso terminará, por lo general debido a una de las siguientes condiciones:



- **Salida normal (voluntaria).** La mayoría de los procesos terminan debido a que concluyeron su trabajo. En este momento ejecutan una llamada al sistema para indicar al sistema operativo que terminó. Esta llamada es *exit* en Unix/Linux y *ExitProcess* en Windows.
- **Salida por error (voluntaria).** El proceso descubre un error , por ejemplo cuando se solicita un archivo que no existe, y termina.
- **Error fatal (involuntaria).** Suele suceder por un error en el programa, como una división por cero.
- **Eliminado por otro proceso (involuntaria).** En Unix/Linux esta llamada es *kill*. La función correspondiente en Win32 es *TerminateProcess*. En ambos casos, el proceso eliminador debe tener la autorización necesaria para realizar la eliminación.

1.5. Tipos de procesos

Podemos clasificar los procesos en función de distintos criterios:

- **Según su diseño:**
 - **Reutilizables:** se cargan en memoria cada vez que se usan. Los programas de usuario suelen ser de este tipo.
 - **Reentrantes:** se carga una sola copia del código en memoria. Cada vez que se usan se crea un nuevo proceso con su zona de datos propia, pero compartiendo el código. Por ejemplo, la ejecución de varios **ls** concurrentes.
- **Según su acceso a CPU y recursos:**
 - **Apropiativos:** acceden a los recursos y sólo los abandonan de forma voluntaria (mediante instrucción CPU).
 - **No apropiativos:** permiten a otros procesos apropiarse de los recursos que ahora poseen
- **Según su permanencia en memoria:**
 - **Residentes:** tienen que permanecer en memoria durante toda su evolución (desde creación hasta terminación).
 - **Intercambiables (swappable):** es lo más normal. El SO puede decidir llevarlos a disco a lo largo de su evolución.
- **Según su propietario:**
 - **Procesos de usuario:** son los diseñados por los usuarios. Se ejecutan en modo no protegido/ no privilegiado.
 - **Procesos del sistema:** son los que forman parte del SO (de E/S, de planificación de otros procesos, etc).

- **Según la manera de ejecutarlos:**

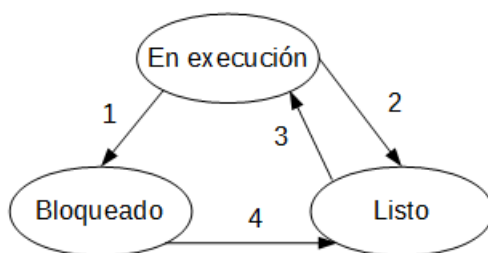
- **Procesos en primer plano:** son procesos que interactúan con el usuario, es decir recibe la información del usuario por un terminal por el que también contesta con los resultados. Sólo puede haber un proceso ejecutándose en primer plano.
- **Procesos en segundo plano:** son procesos que no interactúan con el usuario. Puede haber muchos procesos ejecutándose en segundo plano.

1.6. Estados de un proceso

Un proceso puede estar en diferentes estados en el transcurso de su ciclo de vida. Aunque el número de estados y los nombres que reciben pueden variar entre diferentes sistemas operativos, todos siguen un patrón general .

Un proceso puede encontrarse en los siguientes estados:

- **En ejecución.** El proceso está utilizando el procesador.
- **Listo o preparado.** Ejecutable; el proceso se detuvo temporalmente para dejar que se ejecute otro.
- **Bloqueado.** El proceso no puede ejecutarse hasta que ocurra cierto evento externo. Este estado también se denomina **dormido** o **en espera**.



Este diagrama muestra los tres estados en los que se puede encontrar un proceso y las transiciones entre estos estados (1,2,3,4):

- 1. El proceso se bloquea al solicitar una operación de E/S (entrada/salida) u outro evento y deja de usar el procesador.**
- 2. El planificador selecciona otro proceso.**
- 3. El planificador selecciona este proceso y pasa a utilizar el procesador.**
- 4. La operación de E/S u otro evento, por el que estaba esperando el proceso, ya acabó.**



- La **transición 2** ocurre cuando el planificador decide que el proceso en ejecución se ejecutó el tiempo suficiente y es momento de dejar que otro proceso tenga una parte del tiempo de la CPU.
- La **transición 3** ocurre cuando todos los demás procesos tuvieron su parte del tiempo de la CPU y es momento de que el primer proceso obtenga la CPU para ejecutarse de nuevo.

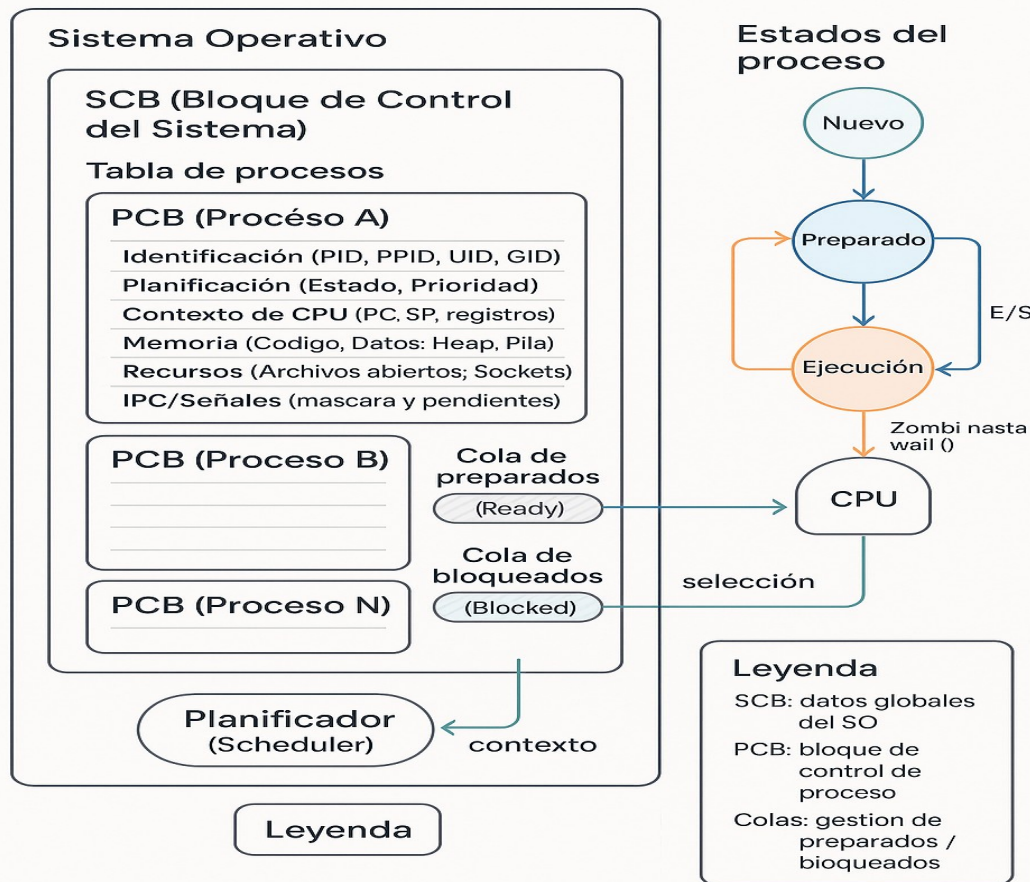
1.7. Implementación de los procesos

Para implementar el modelo de procesos, el sistema operativo necesita saber **la lista de los procesos que tiene y el estado en el que están**.

Estos datos los almacena en el **Bloque de Control del Sistema (SCB)** : conjunto de estructuras de datos que usa el sistema operativo para gestionar la ejecución de los procesos.

Además, el sistema operativo gestiona una tabla, llamada **tabla de procesos**, donde almacena la información relativa a cada proceso. Cada entrada de esta tabla se llama **bloque de control de proceso (PCB)** y contiene los datos particulares de cada proceso que usa el sistema operativo para gestionarlo.

Modelo de procesos (SO)



El PCB normalmente incluye información de:

- **Identificación (ID):** ID del proceso, ID del proceso padre, ID de usuario y grupo
- **Planificación:** estado del proceso , prioridad del proceso
- **Memoria asignada:** punteros a los segmentos de datos, código,pila.
- **Recursos asignados:** archivos abiertos ,...
- **Información para comunicación entre procesos:** señales,

1.8. Concepto de Hilo

Podemos definir un **hilo** , hebra o subproceso, como la unidad de procesamiento más pequeña que puede ser planificada por un sistema operativo. Es una secuencia de instrucciones.

En los sistemas operativos tradicionales cada proceso tiene un espacio de direcciones y un solo hilo (monohilo). Los sistemas operativos actuales

soportan procesos que tiene varios hilos en el mismo espacio de direcciones que se ejecutan en cuasi-paralelo (multihilo).

Las ventajas de los procesos multihilos son las siguientes:

- **Muchas aplicaciones desarrollan varias actividades a la vez.** Al descomponer una aplicación en varios hilos secuenciales que se ejecutan en cuasi-paralelo, el modelo de programación se simplifica.
- **Como los hilos son más ligeros que los procesos,** son mas fáciles de crear (es decir, rápidos) y destruir.
- **Se ejecuta más rápido una aplicación cuando hay una cantidad considerable de cálculos y operaciones de E/S;** al tener hilos estas actividades se pueden solapar.
- **En los sistemas con varias CPUs** es posible el verdadero paralelismo (cada procesador ejecutará un hilo).

Un ejemplo que muestra estas ventajas sería pensar en un usuario que está trabajando con un procesador de textos: **un hilo** interactúa con el usuario que está tecleando texto , **otro hilo** guarda el contenido de la RAM al disco en forma periódica (autoguardado) y **un tercer hilo** podría estar aplicando en el resto del documento un cambio de formato del texto. Los tres hilos modifican el documento ; es decir comparten una memoria común (que si fuesen procesos separados no funcionaría).

Lo que agregan los hilos al modelo de procesos es permitir que se lleven a cabo varias ejecuciones en el mismo entorno del proceso, que son en gran parte independientes unas de las otras. Hay similitudes entre hilos y procesos :

- Los hilos comparten un espacio de direcciones y otros recursos;
- Los procesos comparten la memoria física, los discos y otros recursos.
- Los hilos se ejecutan pseudo (pseudo) concurrentemente dentro de un proceso;
- Los procesos se ejecutan (pseudo) concurrentemente en una computadora.
- El multihilo funciona de la misma manera que la multitarea (multiprogramación): la CPU conmuta de un hilo a otro.

Los hilos se diferencian de los procesos en que no son tan independientes como los procesos. Al compartir más recursos (espacio de direcciones , usuario propietario, archivos abiertos..) es más fácil cambiar de un hilo a otro dentro de un mismo proceso (porque la información a salvaguardar es menor). La comunicación entre hilos es más común que entre procesos.

Al igual que un proceso tradicional (es decir, un proceso con sólo un hilo), un hilo puede estar en uno de varios estados: en ejecución, bloqueado, listo o terminado.

2. LINUX. Gestión de Procesos

- En Linux, **la estructura de procesos es jerárquica**. Para representar la relación entre los procesos se utiliza el concepto de procesos padre-hijo. Por ejemplo, cuando ejecutamos un comando desde el *shell* estamos iniciando otro proceso, que será hijo del proceso padre, el *shell*.
- A cada proceso que se ejecuta en un sistema Linux se le asigna un identificador único llamado **PID** o **identificador de proceso**. El PID es un número entero. Además del PID, los procesos tienen asignado otro número denominado **PPID** que identifica al proceso padre del proceso.
- Cada proceso está asociado con un usuario, el **UID** (número de identificación de usuario), y con uno o más grupo, el **GID** (número de identificación de grupo), que determinan los derechos del proceso para acceder a los recursos del sistema y ficheros.
- En Linux es posible crear un canal de comunicación entre dos procesos; un proceso puede escribir un flujo de bytes para que otro los lea. Estos canales se conocen como *tuberías (pipelines)*. **La tubería en la línea del shell se implementa con el símbolo de tubería |**. Los procesos también se pueden comunicar de otra forma: un proceso puede enviar lo que se conoce como una señal a otro proceso.
- El planificador de procesos utiliza prioridades. El núcleo de Linux utiliza un algoritmo dinámico para calcular las prioridades. También presta atención a un valor establecido administrativamente que suele denominarse "valor **nice**".
- En Linux, los diferentes estados de un proceso se identifican de la siguiente manera:
 - **En ejecución / Running (R)**. El proceso está ejecutándose.
 - **Bloqueado / Durmiendo / Sleeping (S)**.- El proceso está esperando por algún tipo de evento (por ejemplo, la recepción de una señal software o hardware). En cuanto dicho evento se produce, el proceso pasará a ejecutarse/continuará.
 - **Parado / Traced (T)**.- Un proceso parado no está esperando por evento alguno. Se ha detenido su ejecución y sólo pasará a ejecutarse cuando reciba una señal determinada que le permita continuar.
 - **Zombie (Z)**.- Todo proceso al finalizar avisa a su proceso padre, para que éste elimine su entrada de la tabla de procesos. En el caso de que



el padre, por algún motivo, no reciba esta comunicación no lo elimina de la tabla de procesos. En este caso, el proceso hijo queda en estado **zombie, o defunct** no está consumiendo CPU, pero sí continua consumiendo recursos del sistema.

- **(D).** El proceso está en espera/letargo ininterrumpible. (uninterruptible sleep). Este estado suele ser transitorio.
- **Los usuarios ejecutan sus programas:** en el entorno gráfico haciendo doble clic; en el interprete de comandos escribiendo el nombre del fichero ejecutable con sus opciones y argumentos. En este caso hay que tener en cuenta que Linux no incluye el directorio actual en la variable `PATH`, por lo que para ejecutar un programa del directorio actual (que no esta incluido en la variable `PATH`) teclearemos `./nombre_del_programa`.
- **Cada usuario tiene siempre un proceso activo:** el propio shell. Si ejecutamos un comando, estaremos iniciando otro proceso, que será hijo del proceso con el que lo lanzamos.
- En Linux, a los procesos ejecutados por los usuarios desde el shell se les denomina **trabajos o tareas (jobs)**. Así, una tarea es simplemente la unidad de trabajo del shell.
- Un usuario puede ejecutar varios trabajos de forma simultánea. Sin embargo, solo uno de ellos podrá tener la interacción directa con el usuario. A este trabajo se le llama trabajo en **primer plano (o foreground)**. El resto de trabajos puede ejecutarse en lo que se llama **segundo plano (o background)**. Aunque el usuario no puede tener interacción directa con los procesos en segundo plano, si uno de dichos trabajos genera una salida, ésta aparece en el terminal (para evitarlo se deberá redireccionar esta salida). Si el trabajo en segundo plano requiere entradas en tiempo de ejecución, dicho trabajo se quedará parado o en stop.
- **Procesos en primer plano (foreground):** Cuando el *shell* queda a la espera de la finalización del programa / comando ejecutado, y por lo tanto el terminal no acepta nuevos programas / comandos hasta la finalización del proceso hijo.

Para lanzar un proceso en primer plano o foreground, se ejecuta de forma normal.

Ejemplo: `ls -l /var/log > ls_log`

- Una vez ejecutado un proceso en primer plano, no podremos introducir nuevos comandos hasta que termine su ejecución y volvamos a ver el prompt.

- Un programa ejecutándose en primer plano puede **cancelarse** pulsando las teclas **Ctrl-C**. Con esto **matamos el proceso** que está en ejecución.
- También podemos **suspender / parar** el proceso que se está ejecutando en primer plano pulsando las teclas **Ctrl-Z**. Con esto el proceso no termina, si no que pasa al segundo plano en un estado de bloqueo permanente. (muestra por pantalla el nº de trabajo asignado , estado parado)

```
$mousepad  
^Z  
[1]+  Detenido                mousepad  
$
```

Ejemplo de suspender / parar / detener un proceso

- En primer plano se lanzan los programas que no tardan mucho en ejecutarse y los programas que necesitan interacción con el usuario.
- **Procesos en segundo plano (background):** Si el proceso o procesos hijos se ejecutan concurrentemente al proceso *shell* padre, que continúa aceptando nuevos programas/comandos.

Para lanzar un proceso en segundo plano o background, se añade al final del comando el carácter &.

Ejemplo: `ls -l /var/log > ls_log &`

- Al ejecutar en segundo plano el shell imprime una línea, por ejemplo **[1] 6190** y nos devuelve al prompt:
- El nº entre corchetes es el nº de trabajo o tarea que el shell asigna al proceso al pasar a segundo plano (un número entero, comenzando por 1 enumerado secuencialmente)
- El otro nº es el número de proceso asignado, el PID.

```
$mousepad &  
[1] 2237  
$
```

- La única manera de comunicarse con un proceso en segundo plano es mediante el envío de una señal.
- Cuando ejecutamos un programa gráfico (gedit,..etc) desde la terminal, lo normal es lanzarlo en segundo plano, para que podamos ejecutar otros programas/comandos. También se suelen ejecutar en segundo plano los programas cuya ejecución va a tardar bastante tiempo y además no necesitan interaccionar con el usuario.
- También se denominan procesos en modo batch.
- Además de los procesos ejecutados por los usuarios (en primer plano=interactivos o en segundo plano =procesos batch), en un sistema Linux se pueden estar ejecutando otro tipo de procesos:



- **Demonios (Daemons):** procesos que se ejecutan en segundo plano continuamente (es decir, no terminan nunca hasta que el ordenador se apague o se termina la sesión). Muchos se lanzan durante el inicio del sistema y luego esperan a una petición de usuario o sistema indicando que se requiere su servicio.

Los demonios presentan, generalmente, las siguientes características:

- **No disponen de una interfaz ni gráfica ni textual con el usuario**
- **No realizan una interacción con el usuario ni para entrada ni para salida.**
- **El registro de su funcionamiento y la comunicación de errores se realiza sobre ficheros de log** alojados generalmente en /var /log.

Ejemplo: un ordenador que alberga un servidor Web utilizará un demonio para ofrecer el servicio. Otro ejemplo: los demonios de "tareas" programadas que permiten realizar tareas en determinadas circunstancias temporales (por ejemplo todos los viernes) como mantenimiento del sistema.

- **Procesos del Kernel** que los usuarios ni empiezan ni terminan y que tienen poco control sobre ellos.
- Hay procesos lanzados por otros usuarios en otros terminales.

2.1. Comandos para la gestión de procesos

ps

- El comando **ps** (*process status*), sirve para informarnos acerca de los procesos que en ese momento se están ejecutando en el sistema.
- Sintaxis: **ps [opciones]**
- Sin opciones sólo nos muestra los procesos que el usuario actual ejecuta desde el shell correspondiente (es decir desde el terminal actual).
- La sencillez de la sintaxis de *ps* oculta una considerable complejidad, pues *ps* admite tres tipos diferentes de opciones, así como muchas opciones dentro de cada tipo. Los tres tipos de opciones que admite son:
 - Estilo System V → Las opciones van precedidas por un guión -
 - Estilo BSD → Las opciones no llevan guión.
 - Estilo GNU → Utiliza nombres de opciones largas y van precedidas por doble guión -

- **Versión System V.** Las opciones más utilizadas del comando ps son:
 - e muestra información sobre todos los procesos del sistema.
 - A esta opción es idéntica a la opción -e
 - l listado formato largo (muchos detalles de los procesos)
 - f listado formato completo (muchos detalles de los procesos)
 - u *usuario* muestra los procesos que pertenecen a un usuario concreto

Ejemplo: ps -ef

- **Versión BSD.** Las opciones más comunes son:
 - a muestra también los procesos de otros usuario
 - x muestra también los procesos que no tienen terminal asociado
 - l listado formato largo BSD
 - u listado formato orientado al usuario
 - m muestra información relacionada con la memoria
 - U *usuario* muestra los procesos que pertenecen a un usuario concreto

Ejemplo: ps ax

Otro Ejemplo: ps -f -u tty02

- **Versión GNU:** Las opciones más comunes son:
 - help resumen de algunas opciones comunes de ps
 - forest muestra la jerarquía de las relaciones entre los procesos
 - user *usuario* muestra los procesos que pertenecen a un usuario concreto
- Se pueden mezclar las opciones de SystemV y BSD pero hay algunos conflictos. También hay opciones sinónimas (misma funcionalidad)

Ejemplo: Ver todos los procesos

Sintaxis systemV	Sintaxis BSD
ps -e ps -ef ps -efl	ps ax ps aux



```
$ ps -el
F S      UID      PID      PPID      C  PRI   NI   ADDR  SZ  WCHAN  TTY          TIME CMD
4 S      0        1        0  0  80    0 - 13236 -      ?           00:00:01 systemd
1 S      0        2        0  0  80    0 -      0 -      ?           00:00:00 kthreadd
1 S      0        3        2  0  80    0 -      0 -      ?           00:00:00 ksoftirqd/0
1 S      0        5        2  0  60 -20 -      0 -      ?           00:00:00 kworker/0:0H
1 S      0        7        2  0  80    0 -      0 -      ?           00:00:00 rcu_sched
1 S      0        8        2  0  80    0 -      0 -      ?           00:00:00 rcu_bh
```

- El comando *ps* con la opción */* nos muestra las propiedades asociadas a los procesos. El significado de cada uno de los campos es:
 - **S** estado asociado a cada proceso: R , T, S , Z, D. (explicado anteriormente)
 - **UID** nº identificación del usuario propietario del proceso
 - **PID** nº de identificación del proceso
 - **PPID** nº de identificación del proceso padre
 - **C** Indica el uso del procesador por parte del proceso. Es un valor entero del porcentaje de uso del procesador (%CPU).
 - **PRI** prioridad asociada al proceso
 - **NI** valor **nice** empleado. El valor por defecto es 0. Los valores positivos representan la prioridad reducida, mientras que los negativos representan la prioridad incrementada.
 - **ADDR** dirección del proceso en memoria
 - **SZ** Tamaño en bloques del proceso
 - **WCHAN** evento de espera del proceso. Si esta en blanco significa que el proceso se esta ejecutando.
 - **TTY** terminal asociada al proceso. Algunos procesos no están asociados a ningún terminal, en cuyo caso aparece en esta columna el símbolo de interrogación ? .
 - **TIME** Cantidad total de tiempo de procesador consumido.
 - **CMD** Comando que se ejecuta y que da lugar al proceso (COMMAND en otros listados)
- El significado de otros campos que aparecen al ejecutar el comando *ps*:
 - **USER** nombre del usuario propietario del proceso.
 - **%CPU** porcentaje de utilización del procesador por parte del proceso.
 - **%MEM** porcentaje de utilización de la memoria por parte del proceso.
 - **STIME / START** Instante de comienzo del proceso
 - **VSZ / VIRT** Tamaño de la imagen del proceso en memoria virtual (en KiB)

- **RSS / RSZ** Indica la cantidad de kilobytes del programa en memoria residente
- **STAT** estado del proceso
- Dado el enorme número de procesos que se ejecutan simultáneamente en un sistema Linux, resulta muy práctico poder buscar los procesos según algún criterio, por ejemplo los lanzados por un usuario,.. etc. Para esto utilizaremos el comando **grep**: enlazamos la salida del comando **ps** con **grep** para obtener sólo los procesos que contienen un texto determinado.

top

- **ps** nos muestra una "instantánea" de los procesos en el momento que ejecutamos el comando. El comando **top** es la versión interactiva de **ps**.
- El comando **top** muestra, en tiempo real, un listado de los procesos que se están ejecutando en el sistema. También muestra un resumen del estado del sistema.
- Este comando es una buena herramienta para saber cuanto tiempo de CPU consumen unos procesos con respecto a los otros, o ver que procesos consumen más tiempo de CPU. Por defecto, **top** ordena sus entradas por uso de la CPU, actualizando los datos mostrados cada poco segundos.
- Al ejecutar **top** sin opciones presenta una pantalla dividida en tres zonas: área de información del sistema , la línea de mensajes/prompt y la lista de procesos.

```
top - 23:08:23 up 7 min, 4 users, load average: 0,03, 0,16, 0,12
Tasks: 145 total, 1 running, 144 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0,0 us, 0,3 sy, 0,0 ni, 99,7 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
KiB Mem: 1020372 total, 552724 used, 467648 free, 40012 buffers
KiB Swap: 1768444 total, 0 used, 1768444 free. 264144 cached Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1464	root	20	0	247108	52396	27208	S	0,3	5,1	0:01.47	Xorg
1878	root	20	0	316564	20196	15052	S	0,3	2,0	0:00.29	ica
1946	adminis+	20	0	437036	24780	20720	S	0,3	2,4	0:00.33	xfdesktop
2112	adminis+	20	0	25724	2912	2420	R	0,3	0,3	0:00.64	top
2274	root	20	0	0	0	0	S	0,3	0,0	0:00.04	kworker/0:0
1	root	20	0	52936	6728	4816	S	0,0	0,7	0:01.05	systemd
2	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0,0	0,0	0:00.05	ksoftirqd/0
5	root	0	-20	0	0	0	S	0,0	0,0	0:00.00	kworker/0:0H
6	root	20	0	0	0	0	S	0,0	0,0	0:00.00	kworker/u2:0
7	root	20	0	0	0	0	S	0,0	0,0	0:00.12	rcu_sched
8	root	20	0	0	0	0	S	0,0	0,0	0:00.00	rcu_bh

Área de información do sistema

Línea de mensaxes / prompt

Lista de procesos

• Área de información del sistema:

- La primera línea muestra el tiempo de actividad y la carga media del sistema. Muestra: la hora actual, el tiempo que el sistema lleva encendido, el número de usuarios que están conectados y la carga media, en intervalos de 1,5 y 15 minutos respectivamente (el valor 1 indican que el sistema ocupado, pero no sobrecargado).



- La segunda línea muestra el número total de procesos y el número de procesos en cada estado: Ejecución, Durmiendo, Parado y Zombie
- La tercera línea muestra, en porcentajes, el estado de la CPU.
Indica el tiempo de CPU de: procesos de usuario (**us**), procesos de kernel (**sy**), procesos de usuario a más baja prioridad (**ni**), procesos inactivos (**id**), en espera por entrada/salida (**wa**), en interrupciones hardware (**hi**), en interrupciones de software (**si**), en tiempo teal (**st**) del hipervisor de máquina virtual.
- La cuarta línea indica el uso de la memoria física (RAM), clasificada en: memoria total, utilizada, libre y la utilizada por el buffer.
- La quinta línea indica el uso de la memoria virtual/ área de intercambio, clasificada en: memoria total, utilizada, libre y en caché.

• La lista de procesos:

- Al ejecutar el comando *top* sin opciones, muestra una lista de todos los procesos que se están ejecutando en el sistema. Las columnas de información que muestra, son similares a las que muestra el comando *ps*, con la diferencia de que se actualiza periódicamente.

Las columnas que muestra por defecto son:

- USER (USUARIO): usuario propietario del proceso.
- PR: prioridad del proceso. Si pone RT es que se está ejecutando en tiempo real.
- NI: asigna la prioridad. Si tiene un valor bajo (hasta -20) quiere decir que tiene más prioridad que otro con valor alto (hasta 19).
- VIRT: cantidad de memoria virtual utilizada por el proceso.
- RES: cantidad de memoria RAM física que utiliza el proceso.
- SHR: memoria compartida.
- S (ESTADO): estado del proceso.
- %CPU: porcentaje de CPU utilizado desde la última actualización.
- %MEM: porcentaje de memoria física utilizada por el proceso desde la última actualización.
- TIME+ (HORA+): tiempo total de CPU que ha usado el proceso desde su inicio.
- COMMAND: comando utilizado para iniciar el proceso.
- Utilizando la línea de mensajes / prompt podemos ocultar o mostrar las columnas de información, así como seleccionar los procesos a visualizar.

- **La Línea de mensajes / prompt:**

- Desde esta línea podemos introducir los comandos interactivos de `top`. Estos comandos son de un único carácter, aunque algunos solicitarán más información. Pueden realizar acciones muy diversas. Los más utilizados son:
 - **h** o **?** : Muestra una pantalla con la información de ayuda. Tenemos que teclear **q** para salir de esta pantalla.
 - **q** : Salir de `top`.
 - **k** : Envía una señal a un proceso (comando `kill`). Solicita el PID del proceso (por defecto el primero de la lista) y la señal a enviar (por defecto `15/ SIGTERM`).
 - **r** : Cambia la prioridad de un proceso (comando `renice`). Solicita el PID del proceso (por defecto el primer proceso de la lista) y el valor `nice` (por defecto `15/ SIGTERM`).
 - **f** o **F** : Muestra una pantalla que permite seleccionar las columnas de información a mostrar en la lista de procesos. Las columnas marcadas con un asterisco son las seleccionadas para visualizar , con las flechas de dirección arriba y abajo nos movemos por las distintas columnas y pulsando la barra espaciadora seleccionamos, o no, dicha columna (lo indicará la presencia o ausencia del asterisco). También si pulsamos el carácter `s` , estando situados en una columna, haremos que la lista de procesos se muestre ordenada por dicho campo, que se aplicará pulsando `R`
 - **u** o **q** **U** : Seleccionar procesos de un usuario. Solicita el nombre del usuario; pulsando `Enter` selecciona los de todos.
 - **M** : La lista de procesos se ordenan por el uso de la memoria.
 - **P** : La lista de procesos se ordena por el uso de la CPU (opción por defecto).
- El comando `top` también se puede ejecutar desde la línea de comandos, pasándole opciones que modifican su comportamiento. No se utiliza mucho de esta manera.

htop

- El comando **htop** es semejante a `top`, pero más interactivo y funcional.
- Su interfaz también se divide en tres secciones: una cabecera que muestra información del sistema, la lista de procesos y el pie de página muestra información sobre las teclas de función para interactuar con `htop`.

CPU[	0.7%]			Tasks: 94, 136 thr; 1 running		
Mem[	221M/996M]			Load average: 0.09 0.27 0.32		
Swp[	0K/1.69G]			Uptime: 01:20:11		

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
2563	administr	20	0	24396	3768	3132	R	0.7	0.4	0:00.10	htop
1967	administr	20	0	427M	25172	20864	S	0.0	2.5	0:00.35	xfdesktop --displ
1979	administr	20	0	393M	24744	19252	S	0.0	2.4	0:00.63	xfce4-terminal --
1438	root	20	0	245M	55400	27424	S	0.0	5.4	9:36.20	/usr/bin/X :0 -se
1994	administr	20	0	60548	4960	4296	S	0.0	0.5	0:00.12	xscreensaver -no
1898	root	20	0	309M	20332	15188	S	0.0	2.0	0:02.59	/usr/lib/italc/ic
1963	administr	20	0	158M	18368	15480	S	0.0	1.8	0:00.21	xfwm4 --display :
1	root	20	0	52948	6680	4812	S	0.0	0.7	0:01.02	/sbin/init
443	root	20	0	38280	4804	4256	S	0.0	0.5	0:00.19	/lib/systemd/syst
454	root	20	0	45040	4368	2916	S	0.0	0.4	0:00.20	/lib/systemd/syst
1163	systemd-t	20	0	124M	4448	3916	S	0.0	0.4	0:00.00	/lib/systemd/syst
1152	systemd-t	20	0	124M	4448	3916	S	0.0	0.4	0:00.02	/lib/systemd/syst
1229	root	20	0	37080	2688	2252	S	0.0	0.3	0:00.00	/sbin/rpcbind -w
1240	statd	20	0	37280	2840	2252	S	0.0	0.3	0:00.00	/sbin/rpc.statd
1264	root	20	0	23356	200	4	S	0.0	0.0	0:00.00	/usr/sbin/rpc.idm
1278	root	20	0	244M	3248	2440	S	0.0	0.3	0:00.00	/usr/sbin/rsyslog
1279	root	20	0	244M	3248	2440	S	0.0	0.3	0:00.00	/usr/sbin/rsyslog

F1Help	F2Setup	F3Search	F4Filter	F5Tree	F6SortBy	F7Nice	-F8Nice	+F9Kill	F10Quit
--------	---------	----------	----------	--------	----------	--------	---------	---------	---------

ps tree

- Muestra la estructura jerárquica de los procesos que se están ejecutando.
- Sintaxis: **ps tree [opciones]**

Alguna de estas opciones son:

- p** muestra el nº de identificación de los procesos (PID)
- a** muestra los argumentos de los comandos
- u** muestra al usuario propietario del proceso (entre paréntesis)

```
$ps tree
systemd--ModemManager--{gdbus}
                        --{gmain}
                        --NetworkManager--dhclient
                                      --{NetworkManager}
                                      --{gdbus}
                                      --{gmain}
--acpid
--agetty
--anacron
--applet.py--{gmain}
--2*[at-spi-bus-laun--dbus-daemon]
                        --{dconf worker}}
                        --{gdbus}}
                        --{gmain}}

```

Ejemplo *ps tree* (sin opciones)

pgrep

- El comando **pgrep** busca en la lista de procesos, para localizar el PID a partir del nombre del proceso (similar a **ps | grep**).
- Sintaxis: **pgrep [opciones] patrón**

La opción más utilizada es `-u` , que muestra únicamente los procesos que pertenezcan al usuario especificado.

```
$pgrep -u root cups
1263
1342
$
```

Ejemplo pgrep

pidof

- El comando `pidof` encuentra el identificador de proceso de un programa en ejecución. Ejemplo: `pidof getty`

kill

- Sirve para enviar señales a uno o varios procesos. El que envía la señal debe ser el propietario de los procesos o el administrador del sistema.
- La señal es un código numérico, aunque también se representa por una constante. Cada señal indica una acción a realizar sobre el proceso (detenerlo, reanudarlo,...).

Sintaxis: **kill [-señal | -s señal | -n señal] PID**

kill [-señal | -s señal | -n señal] %númeroTrabajo

kill -l [señal]

- Si no se especifica ninguna señal, `kill` envía la señal número 15, `SIGTERM`, `TERM`, al proceso especificado, con intención de terminar su ejecución. Esta señal avisa al proceso que termine por sí mismo, pero el proceso puede ignorarla. Si queremos eliminar el proceso definitivamente, lo mejor es enviarle la señal nº 9, que no se puede ignorar.
- Para especificar la señal podemos usar simplemente *-señal* ou *-s señal* o *-n señal*.
- Para identificar el proceso al que queremos enviarle la señal se suele utilizar el PID (nº de identificación del proceso) pero también podemos utilizar %nº de trabajo (asignado por el bash, que es un nº diferente al PID).
- El comando `kill` con la opción `-l` o `-L` presenta un listado de las señales.

```
$kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS     8) SIGFPE     9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9 56) SIGRTMAX-8 57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX
```




- Alguna de las señales más utilizadas son:

SIGHUP / HUP / 1: Cuelgue de terminal o muerte del proceso controlador

SIGINT / INT / 2 : Interrupción de teclado (Ctrl-C)

SIGKILL / KILL / 9 : Matar inmediatamente

SIGTERM / TERM / 15: Terminar (de forma controlada)

SIGCONT / CONT / 18: Continuar.

SIGSTOP / STOP / 19: Parar (pero preparado para continuar)

SIGTSTP / TSTP / 20 : Stop de teclado (Ctrl-Z)

- La señal se puede especificar de tres maneras diferentes. Ej: -9 -SIGKILL -KILL

```
$jobs -l
[1]+  2494 Parado                sleep 500
$kill -SIGCONT 2494
$jobs -l
[1]+  2494 Ejecutando           sleep 500 &
$kill -9 %1
$jobs -l
[1]+  2494 Terminado (killed)   sleep 500
```

Ejemplos kill

- Existe también el comando: **killall -señal nombre_comando**

Funciona de manera similar a kill, pero con la diferencia de en vez de indicar un PID se indica el nombre del programa, lo que afectará a todos los proceso que tengan ese nombre.

```
$killall -STOP sleep

[1]-  Detenido                sleep 200

[2]+  Detenido                sleep 100
$killall -9 sleep
[1]-  Terminado (killed)     sleep 200
[2]+  Terminado (killed)     sleep 100
```

Ejemplos killall

jobs

- Muestra los trabajos activos. Es decir, los procesos que se ejecutan en segundo plano y que están asociados al shell actual.
- Sintaxis:** jobs [opciones] [id_trabajo]

Opciones:

- l** Visualiza el PID (identificador de proceso) de cada proceso.
- r** Visualiza los procesos que se encuentran en estado de ejecución (running)
- s** Visualiza los procesos que están en estado suspendido (stopped).

Si le pasamos un numero de trabajo solo muestra información sobre ese trabajo.



```
$jobs -l
[1]- 2580 Ejecutando      mousepad &
[2]+ 2584 Parado         sleep 300
$jobs -r
[1]- Ejecutando          mousepad &
$jobs -s
[2]+ Detenido            sleep 300
$jobs
[1]- Ejecutando          mousepad &
[2]+ Detenido            sleep 300
```

Ejemplos ejecución del comando jobs

- La salida del comando es:
 - 1ª columna : indica el número de trabajo asignado (diferente del PID).
 - 2ª columna: puede estar en blanco, o contener un signo más (+), o un signo menos (-):
 - El signo + indica el último trabajo que se ha parado.
 - El signo - indica el penúltimo trabajo que se ha parado.
 - 3ª columna: indica si el trabajo está ejecutándose o se ha parado (Stopped).
 - 4ª columna Contiene el nombre del comando que lanzo el proceso.

bg y fg

- Se utilizan para reanudar un trabajo (tarea) suspendida a partir de su número de trabajo.
- El número de trabajo se puede preceder del signo porcentaje.
- Sintaxis: `bg [id_trabajo]` `fg [id_trabajo]`
 - **bg** reanuda el trabajo en segundo plano. Ejemplo: `bg %1`
 - **fg** reanuda el trabajo en primer plano. Ejemplo: `fg 2`

Sin argumentos actúan sobre el ultimo trabajo suspendido (el que tiene el signo + en la segunda columna de la salida del comando jobs)

```
$jobs -l
[1]- 2634 Parado          sleep 300
[2]+ 2639 Parado          mousepad
$bg %1
[1]- sleep 300 &
$jobs -l
[1]- 2634 Ejecutando      sleep 300 &
[2]+ 2639 Parado          mousepad
$bg 2
[2]+ mousepad &
$jobs -l
[1]- 2634 Ejecutando      sleep 300 &
[2]+ 2639 Ejecutando      mousepad &
$fg %3
[2]- Detenido            mousepad
[3]+ Detenido            sleep 400
$fg %3
sleep 400
```

Ejemplos fg

Ejemplos bg

sleep

- **sleep** especificacion_tiempo.

- Este comando simplemente espera una cantidad de tiempo concreta. La cantidad de tiempo indicada puede ser un entero (segundos) o un entero seguido de la letra s (también segundos), m (minutos), h (horas) o d (días)
- Es útil para retrasar un comando durante una cantidad de tiempo concreta. Suele ser utilizado en los scripts (programas shell). Ejemplo: sleep 10 && echo 'Han pasado 10 seg'.

wait

- **wait** [pid....] .Permite detener un proceso ejecutándose en segundo plano mientras se esta ejecutando un proceso hijo

time

- Sintaxis: **time** [opciones] comando [argumentos]
- Ejecuta el comando especificado y devuelve tiempos relativos a la ejecución del proceso en segundos. Normalmente devuelve tres tiempos:

real : tiempo transcurrido en la ejecución total

user: tiempo que consume el proceso ejecutando su propio código

sys : tiempo que se ha empleado Unix al servicio de la orden.

nice

- En Linux la prioridad de los procesos es gestionada automáticamente por el *Kernel*, pero permite modificar estas prioridades al usuario mediante el número *nice*. Por defecto, los procesos poseen un valor de *nice* igual a cero, y con este valor el kernel no modifica la prioridad del proceso.
- El valor **nice** es un número comprendido entre -20 (prioridad más alta) y 19 (prioridad más baja). Sólo root puede asignar números negativos a nice e incrementar la prioridad. Cualquier usuario puede aumentar el valor de nice y bajar la prioridad del proceso.
- El comando nice permite ejecutar un programa con una prioridad distinta de la normal.
- Sintaxis: **nice** [-n *numero_nice*] [comando] [argumentos]
nice [-*numero_nice*] [comando] [argumentos] .
- Si ejecutamos *nice* sin comando muestra el valor nice actual.
- Si ejecutamos *nice* pasándole un comando, pero sin indicarle el número nice, el comando nice utilizará el valor 10.



```
$nice
0
$nice sleep 2000 &
[2] 2912
$nice -5 sleep 500 &
[3] 2922
```

Ejemplos nice

```
#nice --5 sleep 600 &
[3] 2963
#nice -n -5 sleep 600 &
[4] 2964
#
```

Ejemplos nice

- Existe también el comando: **renice** que permite alterar la prioridad de un proceso sin necesidad de detener el proceso.

renice

- Sintaxis: **renice** [+ | - *numero_nice*] [-p *pid*] [-u *user*] [-g *grp*]
- Donde:
 - p *pid* Cambia la prioridad al proceso con el PID especificado.
 - u *user* Cambia la prioridad para los procesos del usuario especificado.
 - g *grp* Cambia la prioridad para los procesos ejecutados por los usuarios que pertenezcan al grupo cuyo ID sea *grp*.

```
$mousepad &
[1] 3200
$renice 10 -p 3200
3200 (process ID) old priority 0, new priority 10
$
```

Ejemplo renice

nohup

- En Linux, cuando salimos de un *login shell (logout)* o cerramos una terminal, se envía una señal **SIGHUP** a todos los procesos hijo. Si lanzamos un proceso en segundo plano y salimos de la sesión, el proceso muere (termina) al morir el shell desde el que lo iniciamos.
- El comando *nohup* permite ejecutar procesos en segundo plano de manera que ignoren las señales **SIGHUP**. La salida del comando se redirige al fichero *nohup.out*.
- El comando *nohup* permite ejecutar un proceso en segundo plano (terminar con &) de manera que continúe en ejecución después de cerrar sesión.
- Sintaxis: **nohup** comando [argumentos] . Ejemplo: **nohup sleep 2000 &**

```
$nohup sleep 2000 &
[2] 3505
$nohup: se descarta la entrada y se añade la salida a «nohup.out»
$
```

Ejemplo nohup

**nice**

- Permite cambiar la prioridad de un proceso. Por defecto, todos los procesos tienen una prioridad igual ante el CPU que es de 0. Con nice es posible iniciar un programa (proceso) con la prioridad modificada, más alta o más baja según se requiera.
- Las prioridades van de -20 (la más alta) a 19 la más baja. Solo root o el superusuario puede establecer prioridades negativas que son más altas. Con la opción -l de ps es posible observar la columna NI que muestra este valor.

```
#> nice          (sin argumentos, devuelve la prioridad por defecto )
```

```
0
```

```
#> nice -n -5 comando (inicia comando con una prioridad de -5, lo que le da más tiempo de cpu)
```

renice

- Así como nice establece la prioridad de un proceso cuando se inicia su ejecución, renice permite alterarla en tiempo real, sin necesidad de detener el proceso.

```
#> nice -n -5 yes  (se ejecuta el programa 'yes' con prioridad -5)
                        (dejar ejecutando 'yes' y en otra terminal se analiza con 'ps')
```

```
#> ps -el
```

```
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0 12826 12208  4  75  -5 -  708 write_ pts/2   00:00:00 yes
```

```
#> renice 7 12826
```

```
12826: prioridad antigua -5, nueva prioridad 7
```

```
#> ps -el
```

```
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0 12826 12208  4  87   7 -  708 write_ pts/2   00:00:15 yes
```

(obsérvese el campo NI en el primer caso en -5, y en el segundo con renice quedó en 7, en tiempo real)

nohup y &

- Cuando se trata ejecutar procesos en background (segundo plano) se utiliza el comando **nohup** o el operador **&** . Aunque realizan una función similar, no son lo mismo.
- Si se desea liberar la terminal de un programa que se espera durará un tiempo considerable ejecutándose, entonces se usa . Esto funciona mejor cuando el resultado del proceso no es necesario mandarlo a la salida estándar (stdin), como por ejemplo cuando se ejecuta un respaldo o se abre un programa Xwindow desde la consola o terminal. Para lograr esto basta con escribir el comando en cuestión y agregar al final el símbolo & (ampersand).

```
$> yes > /dev/null &
```

```
$> tar czf respaldo /documentos/* > /dev/null/ &
```

```
$> konqueror & (con estos ejemplos se ejecuta el comando y se libera la terminal regresando el prompt)
```

- Sin embargo lo anterior produce que el padre del proceso PPID que se invocó sea el proceso de la terminal en si, por lo que si cerramos la terminal o salimos de la sesión también se terminaran los procesos hijos que dependan de la terminal, no muy conveniente si se desea que el proceso continúe en ejecución.
- Para solucionar lo anterior, entonces se usa el comando nohup que permite al igual que '&' mandar el proceso y background y que este quede inmune a los hangups (de ahí su nombre nohup) que es cuando se cuelga o termina la terminal o consola de la cual se ejecutó el proceso.

```
$> nohup yes > /dev/null &
```

```
$> nohup czf respaldo /documentos/* > /dev/null/
```

```
$> nohup konqueror
```

Asi se evita que el proceso se "cuelgue" al cerrar la consola.

jobs

- Si por ejemplo, se tiene acceso a una única consola o terminal, y se tienen que ejecutar varios comandos que se ejecutarán por largo tiempo, se pueden entonces como ya se vió previamente con nohup y el operador '&' mandarlos a segundo plano o background con el objeto de liberar la terminal y continuar trabajando.



- Pero si solo se está en una terminal esto puede ser difícil de controlar, y para eos tenemos el comando jobs que lista los procesos actuales en ejecución:

```
#> yes > /dev/null &
[1] 26837
#> ls -laR > archivos.txt &
[2] 26854
#> jobs
[1]-  Running          yes >/dev/null &
[2]+  Running          ls --color=tty -laR / >archivos.txt &
```

- En el ejemplo previo, se ejecutó el comando yes y se envió a background (&) y el sistema devolvió [1] 26837, indicando así que se trata del trabajo o de la tarea [1] y su PID, lo mismo con la segunda tarea que es un listado recursivo desde la raíz y enviado a un archivo, esta es la segunda tarea.
- Con los comandos fg (foreground) y bg background es posible manipular procesos que estén suspendidos temporalmente, ya sea porque se les envió una señal de suspensión como STOP (20) o porque al estarlos ejecutando se presionó ctrl-Z. Entonces para reanudar su ejecución en primer plano usaríamos fg:

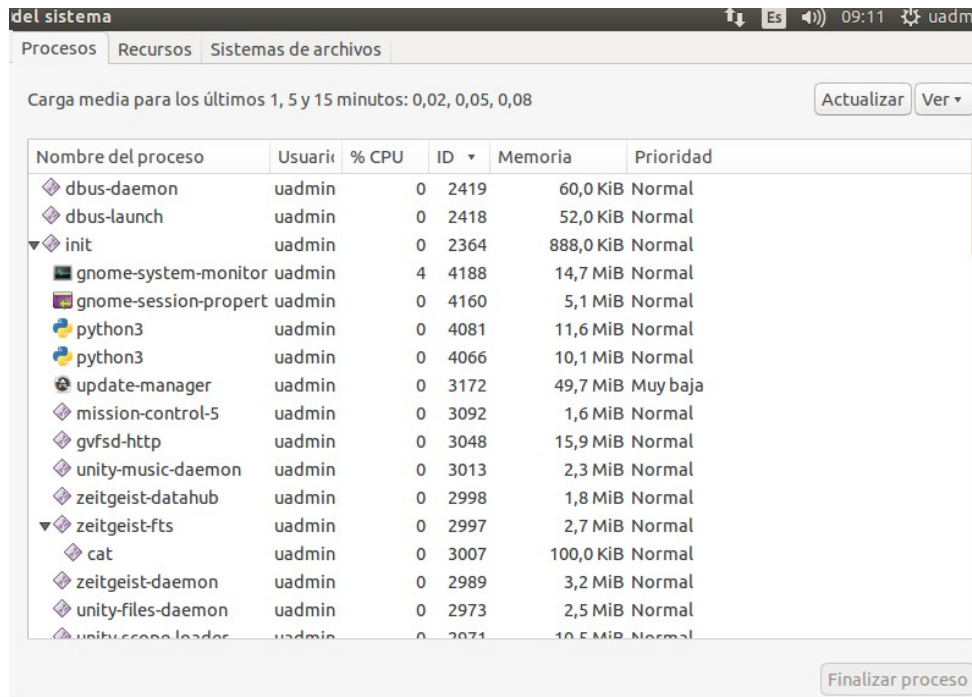
```
#> jobs
[1]-  Stopped          yes >/dev/null &
[2]+  Stopped          ls --color=tty -laR / >archivos.txt &
#> fg %1
#> jobs
[1]+  Running          yes >/dev/null &
[2]-  Stopped          ls --color=tty -laR / >archivos.txt &
```

Obsérvese como al traer en primer plano al 'job' o proceso 1, este adquirió el símbolo [+] que indica que está al frente. Lo mismo sería con bg que volvería a reiniciar el proceso pero en segundo plano. Y también es posible matar los procesos con kill indicando el número que devuelve jobs: kill %1, terminaría con el proceso en jobs número 1.

2.2. Gestión de procesos en el entorno gráfico.

- La aplicación del entorno gráfico que permite ver /manejar los procesos es el **Monitor del sistema**. También monitoriza el estado del sistema mostrando cuanto procesador, memoria y espacio del disco se está usando.

- El Monitor del sistema se puede ejecutar desde la línea de comandos mediante el comando `gnome-system-monitor`.



- Pulsando en la ficha **Procesos** se mostrarán los procesos.
 - En el botón **Ver** podremos elegir entre ver todos los procesos, los activos o los procesos del usuario actual. Ver las dependencias.
 - Se puede seleccionar los campos de información que se muestran sobre cada proceso → Botón derecho sobre el encabezado → Seleccionar/deseleccionar campos (Nombre del proceso, Estado, %CPU,.....)
 - Pulsando el botón derecho sobre un proceso podré detener el proceso, continuar, finalizar, matar...
- En el Monitor del sistema también podré ver:
 - En la ficha **Recursos** se muestra una pantalla mostrando información sobre el uso de la CPU, de la memoria y de la red
 - En la ficha **Sistemas de archivos** información diversa sobre el sistema de archivos.

2.3. El directorio virtual `/proc`

`/proc` es un pseudo-sistema de ficheros que guarda información sobre el sistema y los procesos. Tiene las siguientes características:

- Se inicializa durante el arranque
- Está implementado en memoria y no se guarda en disco



- La estructura del directorio **/proc** depende de la versión del kernel
- Los comandos de procesos (ps, top, etc.) obtienen la información sobre los procesos de este directorio
- El concepto básico es que para cada proceso del sistema se crea un directorio en **/proc**. **El nombre del directorio es el PID del proceso.** En este directorio hay archivos que contienen información sobre el proceso. Por ejemplo, /proc/619 es el directorio que corresponde al proceso con el PID 619.

Alguno de los archivos de estos directorios son:

- **cmdline** Fichero que contiene la línea de comandos con la que fue invocado el proceso.
- **cwd** Es un enlace simbólico al directorio actual del proceso.
- **status** Fichero que contiene el estado del proceso.
- **exe** Es un enlace al nombre del ejecutable.
- **environ** Es un fichero que contiene las variables del entorno del proceso.
- **fd** Directorio que contiene los descriptores de ficheros abiertos, archivos o procesos relacionados.
- También hay otros ficheros y directorios en **/proc** que contienen una amplia variedad de información sobre la CPU, las particiones del disco, los dispositivos, los vectores de interrupción, los contadores del kernel, los sistemas de archivos, los módulos cargados y mucho más. Alguno de estos ficheros y directorios son:
 - **cpuinfo**: información estática de la CPU/Procesador
 - **meminfo**: información de uso de la memoria
 - **partitions**: información sobre las particiones
 - **filesystems**: sistemas de ficheros soportados por el kernel
 - **version**: versión y fecha del kernel
 - **bus/**: directorio con información de los buses PCI y USB
 - **cmdline**: línea de arranque del kernel
 - **devices**: dispositivos del sistema de caracteres o bloques
 - **modules**: módulos del kernel
 - **net/**: directorio con información de red
 - **interrupts**: muestra el número de interrupciones por IRQ
 - **ioports**: lista los puertos de entrada salida usados en el sistema
- Los programas de usuario sin privilegios pueden leer gran parte de esta información para aprender sobre el comportamiento del sistema de una forma segura. Se puede escribir en alguno de estos archivos para modificar los parámetros del sistema.

2.4. Permisos especiales ficheros

Ya sabemos que:

- **Los permisos básicos de un fichero/directorio son : r,w,x.** . Y que se establecen para el propietario, el grupo y para el resto de usuarios(un fichero tiene un usuario propietario y un grupo propietario).
- **Los procesos tienen un usuario propietario y un grupo propietario.** Con los permisos normales, éstos serán, respectivamente, el usuario y grupo principal del usuario que lanzó la ejecución.

Los ficheros pueden tener permisos especiales que afectan a cómo se ejecutan, y por tanto a los procesos. Estos son:

suid (SUID bit) → chmod u+s fichero

- **Para ejecutables** → cambio de dominio a nivel de usuario. Durante la ejecución el usuario efectivo del proceso es el propietario del fichero y no el usuario real que lanzó el proceso.
- También se puede establecer el bit SUID utilizando la notación numérica, añadiendo un cuarto número en la parte izquierda → el bit SUID tiene el valor 4 (4000)
- Se reconoce al hacer un `ls -l` : aparece "s" en lugar de "x" en los permisos de usuario. Si aparece una S es que no tiene el permiso de ejecución y en este caso el permiso no tiene efecto.
- Ejemplo de utilidad:

```
* El ejecutable "gestorbd" lee el fichero "basedatos":
  _rwxr_xr_x  root root /opt/bin/gestorbd
  _rw_____  root root /opt/datos/basedatos
* El usuario pilar puede ejecutar "gestorbd" pero NO leer "basedatos"
* Pilar SÍ podrá leer "basedatos" si "gestorbd" tiene los permisos:
  _rwsr_xr_x  root root /opt/bin/gestorbd
```

Finalidad: cuando un ejecutable tiene **SUID**, al ejecutarlo el proceso toma los privilegios del propietario del archivo (normalmente root), no los del usuario que lo lanza.

- **Uso típico:** permitir a usuarios realizar tareas que requieren privilegios puntuales sin darles root completo.
- **Ejemplo clásico:** `/usr/bin/passwd` (propietario root, con SUID) puede modificar `/etc/shadow`.

sgid (SGID bit) → chmod g+s fichero

- **Para ejecutables** → cambio de dominio a nivel de grupo. Durante la ejecución el grupo efectivo del proceso es el grupo del fichero y no el grupo real del usuario que lo ejecutó.
- Ejemplo de utilidad:

```
* El ejecutable "gestorbd" lee el fichero "basedatos":
  _rwxr_xr_x  root root /opt/bin/gestorbd
  _rw_rw_____ root root /opt/datos/basedatos
* Los miembros del grupo "alumnos" pueden ejecutar "gestorbd"
  pero NO leer "basedatos"
* Ahora SÍ podrán leer "basedatos" con los permisos:
  _rwxr_sr_x  root root /opt/bin/gestorbd
```

- **Para directorios** → al crear un fichero en su interior, el grupo propietario del nuevo fichero es el grupo del directorio y no del usuario que ejecuta la orden.

```
* Si el usuario "pilar" sólo pertenece al grupo "profesor"
* Se tiene el directorio "drwxrwsr_x  pilar alumnos /practicass"
* Si "pilar" ejecuta "cp tema2.pdf /practicass" entonces el
  fichero copiado pertenecerá al grupo "alumnos":
  rw_r_r_  pilar alumnos /practicass/tema2.pdf
```

- Ejemplo de utilidad:
 - También se puede establecer el bit SGID utilizando la notación numérica, añadiendo un cuarto número en la parte izquierda → el bit SGID tiene el valor 2 (2000)
 - Se reconoce al hacer un `ls -l` : aparece "s" en lugar de "x" en los permisos de grupo.

Finalidad en ejecutables: el proceso hereda el GID del archivo al ejecutarse, útil para compartir recursos de grupo.

Finalidad en directorios: los nuevos archivos/directorios creados dentro heredan el GID del directorio, facilitando colaboración en proyectos compartidos.

Uso típico:

- **Ejecutable:** herramientas que necesitan acceso a recursos de un grupo específico.
- **Directorio de trabajo compartido:** mantener todos los ficheros con el mismo grupo.

Directorio de proyecto: `chmod g+s /proyectos/equipo` para que todo lo creado pertenezca al grupo del directorio.



t (sticky bit) → `chmod +t fichero`

- **Para ejecutables** → Mantener la imagen del fichero en memoria después de finalizar la ejecución del mismo. ***Hoy en día no se usa para esta función***
- **Para directorios** → Si tienes permiso de escritura en el directorio, puedes crear ficheros pero sólo puedes borrar los que te pertenecen
 - El directorio `/tmp` tiene los permisos `drwxrwxrwx`
 - Sólo tiene sentido si tiene permisos de escritura para todos (`drwxrwxrwx`)
- También se puede establecer el sticky bit utilizando la notación numérica, añadiendo un cuarto número en la parte izquierda → `1xxx`
- Se reconoce al hacer un `ls -l` : aparece `"t"`

Finalidad (en directorios): sólo el propietario del archivo (o root) puede borrar o renombrar sus archivos dentro del directorio, aunque otros tengan permisos de escritura en ese directorio.

- **Uso típico:** directorios públicos y compartidos donde muchos usuarios escriben

Más Referencias:

<https://es.wikipedia.org/wiki/Setuid>

https://es.wikipedia.org/wiki/Sticky_bit

RESUMEN

- **SUID:** ejecuta con UID del propietario (privilegios del owner). Seguridad: alto riesgo si mal usado.
- **SGID:** ejecuta con GID del archivo; en directorios, hereda GID para nuevos ficheros. Ideal para colaboración.
- **Sticky:** protege archivos en directorios compartidos; sólo su dueño puede borrarlos.

2.5. Otros Comandos - Rendimiento del sistema

Además de ps y top existen comandos básicos que nos pueden mostrar el estado del sistema en cuanto a uso de CPU y consumo de memoria:

- **uptime**: Muestra la hora actual, el tiempo que el sistema lleva encendido, el número de usuarios conectados y la carga media del sistema para los últimos 1, 5, y 15 minutos (lo mismo que en la primera línea de top).

Ejemplo:

```
$ uptime
11:28:43 up 13 min,  5 users,  load average: 0,00, 0,03, 0,05
```

- **w**: Además de la información dada por uptime, el comando w muestra información sobre los usuarios y sus procesos

Ejemplo:

```
$ w
11:29:24 up 14 min,  5 users,  load average: 0,00, 0,02, 0,04
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU WHAT
pepe      tty1                11:19    9:16   0.08s   0.05s -bash
administ  :0        :0              11:17    ?xdm?  6.93s   0.00s /bin/sh /etc/xd
administ  pts/0     :0              11:18    11:22   0.03s   0.03s bash
administ  pts/1     :0              11:18    0.00s   0.03s   0.00s w
alumno    tty2                11:20    9:06   0.08s   0.04s -bash
```

Definiciones:

- LOGIN la hora a la que se conectó el usuario
- IDLE tiempo que lleva ocioso el terminal
- JCPU el tiempo de CPU consumido por los procesos que se ejecutan en el TTY
- PCPU tiempo consumido por el proceso actual (el que aparece en la columna WHAT)
- **free**: Muestra la cantidad de memoria libre y usada en el sistema, tanto para la memoria física como para el swap, así como los buffers usados por el kernel

(similar a lo mostrado en la cabecera de top)

Ejemplo:

```
$ free
              total        used        free      shared    buffers     cached
Mem:          1020372       577120       443252         8648       43064       265292
-/+ buffers/cache:       268764       751608
Swap:         1768444              0       1768444
```

- La columna shared no significa nada (obsoleta)
- Opciones:

- -b,-k,-m,-g memoria en bytes/KBytes/MBytes/GBytes
 - -t muestra una línea con el total de memoria (física + swap)
 - -s delay muestra la memoria de forma continua, cada delay segundos
- Se puede instalar el paquete **sysstat** que contiene una colección de herramientas de monitorización de rendimiento. Pueden proporcionar datos instantáneos de rendimiento o almacenarlos como históricos para futuras consultas. Especialmente en entornos de servidor, sus datos proporcionan información muy valiosa sobre posibles cuellos de botella de nuestro sistema. Algunas de las herramientas que incluye son:
 - **iostat**: Estadísticas de CPU, de dispositivos de E/S, de particiones y del sistema de archivos
 - **vmstat**: Informa sobre uso de la memoria física y virtual,
 - **mpstat**: Estadísticas del rendimiento de cada procesador del sistema
 - **pidstat**: Estadísticas de los procesos de Linux
 - **sar**: Recopila, informa y guarda la información de actividad del sistema (CPU, memoria, discos, interrupciones, interfaces de red, TTY, tablas de kernel, etc.)

last / lastb /lastlog → Mostrar últimos accesos / accesos fallidos / última conexión por usuario

Recordar comandos **df** , **du** (visto año pasado) para averiguar el rendimiento del disco

3. LINUX. El proceso de arranque – Gestión de servicios

Siempre que se modifica el modo en que arranca el ordenador, existe la posibilidad de no obtener los resultados esperados. En estos casos, es útil saber dónde encontrar más información sobre lo que ha ocurrido durante el inicio.

3.1. El proceso de arranque - etapas

El **proceso de arranque** de Linux es el proceso que inicia el sistema operativo cuando el usuario enciende el ordenador. Se encarga de la inicialización del sistema y de los dispositivos. Consiste en todo lo que ocurre desde que se enciende el ordenador hasta que la interfaz de usuario es plenamente operativa.

Entender los pasos del proceso de inicio ayudará a solucionar problema y a ajustar el rendimiento del ordenador.

Las etapas básicas del proceso de arranque para un sistema x86 son:



1. Arranque del Hardware:

Se enciende el sistema, con lo que un circuito de hardware especial hace que la CPU mire en una determinada dirección y ejecute el código almacenado en dicha ubicación. En ésta se encuentra el firmware (BIOS o EFI), por lo que la CPU lo ejecuta.

El firmware realiza algunas tareas entre las que se incluyen la comprobación del hardware (POST), su configuración y buscar un gestor de arranque.

2. Gestor de arranque (Boot Loader). El control pasa del firmware al gestor de arranque.

El gestor de arranque es normalmente almacenado en uno de los discos duros del sistema, bien en el sector de arranque (para sistemas tradicionales BIOS/MBR) o en la partición ESP (para sistemas EFI-UEFI).

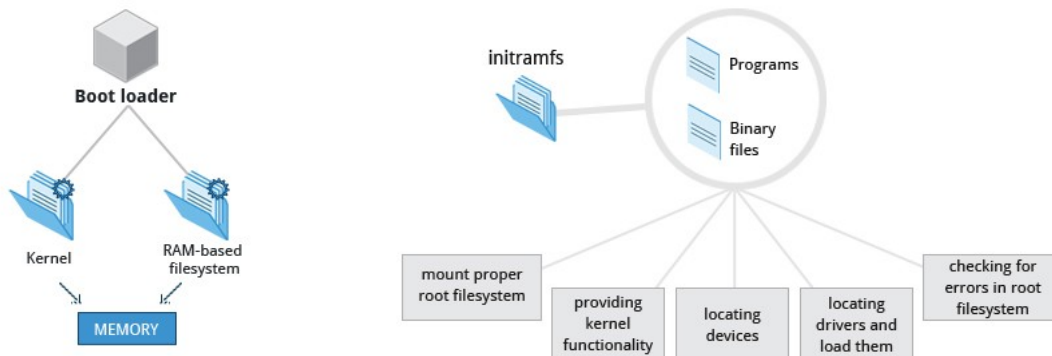
En Linux, el gestor de arranque más utilizado es GRUB. Presenta una pantalla al usuario para elegir el sistema operativo o kernel con el que se desea arrancar. Después de escoger el sistema operativo, el gestor de arranque carga el kernel seleccionado, ejecutando la orden `/boot/vmlinuz-<kernel-version>`, y un disco/sistema de ficheros-RAM inicial (`initramfs`) a memoria.

3. El Kernel de Linux + `initramfs`.

Cuando el **kernel** de Linux toma el control, inicializa y configura la memoria del ordenador y también configura el hardware (los procesadores, los dispositivos de almacenamiento,..etc). También carga algunas aplicaciones de usuario necesarias.

El **`initramfs`** contiene programas y archivos binarios que realizan varias tareas cuyo propósito es montar la partición raíz. Se encarga de cargar los drivers de los dispositivos de almacenamiento que no están compilados en el kernel, mediante la utilidad **`udev`**. Sólo cargará los módulos necesarios para acceder a la partición raíz; el resto se cargarán más tarde durante la fase `init`

El programa **`mount`** informa al sistema operativo de los sistemas de ficheros que están listos para usar, y los asocia con un punto de montaje . Si esto es correcto, el **`initramfs`** se elimina de la RAM y se carga y ejecuta el programa inicial del sistema, que por defecto es `/sbin/init`.



4.El proceso de inicialización /sbin/init y servicios.

El programa `/sbin/init` (también llamado `init`) recibe el ID de proceso (PID) 1, puesto que es el primer programa que se ejecuta en el sistema. Éste iniciará otros procesos para conseguir que el sistema funcione (será el origen de todos los procesos del sistema, excepto los procesos kernel).

Un sistema Linux cuenta con numerosos programas ejecutados de fondo para proporcionar servicios al sistema; son los que en terminología Linux se llaman demonios (daemons). El programa `init` se encarga de iniciar todos estos programas al iniciar Linux. Es el denominado proceso de inicialización.

El programa `init` original de Linux se basa en el de **System V** (SysV), que utiliza una serie de scripts divididos en varios niveles de ejecución para determinar qué programa ejecutar en cada momento.

El programa `init` de SysV se utilizó durante muchos años, pero al aumentar la complejidad de los sistemas, y la necesidad de nuevos servicios, los scripts de nivel de ejecución se complicaron, lo que llevó a buscar soluciones alternativas.

En la actualidad la mayoría de los sistemas Linux reemplazaron el `init` SystemV por el proceso de inicialización **systemd**. El primer programa que se ejecuta, con PID 1, es `systemd`. Su principal objetivo es controlar el contorno dinámico generado en Linux por los dispositivos de conexión en caliente.

5.Login en modo texto.

Cerca del fin del proceso de arranque, **init** inicia un número de indicadores de inicio de sesión en modo texto (hecho por un programa llamado **getty**). Estos permiten introducir un nombre de usuario y una contraseña e iniciar una shell (generalmente **bash**) a la espera de la introducción de comandos.



La mayoría de las distribuciones disponen de seis terminales de texto, a las que se accede ALT + F1, ALT+F2,... y una terminal de gráfico. (CTRL + ALT + F1,... para acceder a las terminales)

6.Sistemas de Ventanas X

Generalmente, en un Linux de escritorio, el Sistema de Ventanas X está cargado como el paso final en el proceso de arranque. Un servicio llamado **gestor de pantalla (display manager)** carga el servidor X (así llamado porque proporciona servicios gráficos a aplicaciones, llamadas clientes X). También maneja los **logins gráficos** y ejecuta el apropiado entorno de escritorio después del login del usuario.

Si el **gestor de pantalla** no se inicia por defecto, después del login en una consola modo-texto, ejecutamos el comando **startx**.

3.2. Extraer información sobre el proceso de arranque

Hay cierta información del kernel y del registro de módulos (para que el núcleo no tenga un tamaño muy grande, la mayoría de las opciones se compilan como módulos, que se cargarán cuando se necesiten), que se almacena en lo que se denomina búffer circular del kernel. Por defecto, Linux muestra los mensajes destinados al búffer circular durante el proceso de arranque; son esos mensajes que se desplazan por la pantalla demasiado rápido como para leerlos. Podemos inspeccionar esta información con el comando **dmesg** : la opción -H lo visualiza en formato humano (tiene más opciones ...)

Otra fuente de información registrada es la generada por el demonio de registro del sistema **rsyslogd** (es una mejora de **syslogd**). Los archivos de registro contiene mensajes de actividad del s.o (del núcleo, de los servicios y aplicaciones que se ejecutan). Estos archivos de *log*, están en el subdirectorio /var/log y el registro de **rsyslog** suele encontrarse en /var/log/syslog : contiene la mayor cantidad de información por defecto del s.o. Otros archivos de registro de /var/log también pueden contener información útil, pues algunos servicios escriben información de registro directamente en sus archivos de registro.

Los mensajes de arranque del búfer circular o de los archivos /var/log pueden generar una abundante salida, por lo que es recomendable canalizarla a través del paginador less o redirigirla a un archivo: **dmesg | less** o **dmesg> boot.mensajes**

Mirar !!!

[Cómo usar Journalctl para ver y manipular los registros de Systemd | DigitalOcean](#)



3.3. El proceso de inicialización System V

Se debe configurar el proceso de inicialización para iniciar programas en función de las prestaciones concretas que deseamos ejecutar en un sistema. Por ejemplo, un servidor Linux no siempre necesita iniciar un entorno de escritorio gráfico o un escritorio Linux no siempre necesita iniciar el servicio de servidor Web Apache.

El proceso de inicialización SystemV, también denominado SysV, es el tradicional de Linux (heredado de System V de UNIX).

La clave del proceso de inicialización SysV son los modos /niveles de ejecución. El programa init determina qué servicios / demonios iniciar en función del nivel de ejecución del sistema.

Los niveles de ejecución están numerados del 0 al 6 y cada uno tiene asignado un conjunto de servicios /demonios que deberían estar activos. Al iniciarse, Linux pasa a un nivel de ejecución predeterminado, que se puede configurar.

La siguiente tabla sintetiza los usos convencionales de los niveles de ejecución:

Nivel de ejecución	Finalidad
0	El nivel de ejecución 0 está configurado para cerrar/parar el sistema. En equipos con hardware moderno, el sistema se apaga por completo.
1, s o S	Modo monousuario. Se inician los demonios/servicios mínimos para realizar tareas de mantenimiento (por ejemplo redimensionado de las particiones) . Habilita una única terminal para root.
2 a 5	Niveles destinados a ser configurados según las necesidades de cada distribución Linux. Habitualmente todos son multiusuario. En algunas distribuciones el nivel 2 está configurado como multiusuario sin servicio de red, el nivel 3 como multiusuario con servicio de red y el nivel 5 para poner en marcha el sistema con entorno gráfico.
6	Este nivel se emplea para reiniciar el sistema. El sistema se apaga por completo y el ordenador se reinicia automáticamente.

Cada nivel de ejecución tiene asociado un directorio **/etc/rc?.d** (donde ? es el número del nivel de ejecución). En estos directorios encontramos enlaces simbólicos a los scripts que controlan los demonios/servicios, que están situados en /etc/init.d/.

Vemos por ejemplo el contenido del directorio asociado al nivel de

```
$ls /etc/rc1.d/
K01alsa-utils      K01lwsmd           K01winbind         S01bootlogs
K01atd             K01network-manager K02avahi-daemon    S01killprocs
K01cups-browsed    K01nmbd            K02cups            S01motd
K01exim4           K01samba-ad-dc     K04rsyslog         S02single
K01hddtemp         K01saned           K06nfs-common      K06rpcbind
K01irqbalance     K01smbd            K06rpcbind
K01lightdm         K01speech-dispatcher README
```

ejecución 1:

La primera letra del nombre de estos enlaces simbólicos indica la acción a realizar:

- K: si el enlace comienza por K (kill) indica detener el demonio.
- S: si el enlace comienza por S (start) indica iniciar el demonio.

Después de esta letra hay un número de dos dígitos, entre 00 y 99, que indica el orden en que se ejecutan los scripts. Este orden es importante, ya que algunos demonios necesitan que otros estén en ejecución antes de poder iniciarse. Finalmente aparece el nombre del demonio en cuestión que interesa iniciar o detener.

SystemV - Gestión de Servicios

Así, aunque un demonio/servicio es un proceso como cualquier otro que se ejecuta en segundo plano, el modo de gestionarlo e invocarlo es diferente al resto de ordenes y programas del sistema. Generalmente, los demonios tienen un script (también llamado guión de shell o shell script) situado en el directorio `/etc/init.d` que permite iniciarlos, pararlos o ver su estado de ejecución siguiendo la **sintaxis**:

`/etc/init.d/nombre_del_servicio acción`

Los parámetros de acción básicos que debe aceptar el script del demonio son:

- **start**: Iniciar el demonio
- **stop**: Parar el demonio
- **restart**: Reinicia el demonio y sirve para que se vuelvan a leer sus archivos de configuración.
- **reload**: aunque no todos los demonios lo permiten, esta acción permite recargar los archivos de configuración sin tener que parar el demonio.
- **status** : Mostrar el estado actual

Por ejemplo, para iniciar el servidor Samba ejecutaríamos:

`/etc/init.d/smb start`

El directorio `/etc/default` contiene archivos que definen variables de configuración del demonio.

Muchas distribuciones también disponen del comando `service` para iniciar/detener .. los servicios. Sintaxis:

`service nombre_del_servicio acción`

Ejemplo: `# service atd status`

update-rc.d

- La modificación de la configuración de un nivel de ejecución se puede hacer manualmente: incluyendo el script de tratamiento del servicio en el directorio `/etc/init.d` y creando los enlaces simbólicos en cada directorio del nivel de ejecución. También se puede hacer de manera más cómoda en Debian y derivados con el comando `update-rc.d` que



crea automáticamente los enlaces simbólicos necesarios para cada nivel de ejecución (en otras distribuciones puede cambiar el comando).

- El comando **update-rc.d** activa/desactiva servicios para niveles de ejecución específicos
- Sintaxis: `update-rc.d [-n] [-f] <basename> remove`
`update-rc.d [-n] <basename> defaults`
`update-rc.d [-n] <basename> disable | enable [S|2|3|4|5]`

La opción **-f** indica que fuerce la eliminación y la opción **-n** no hace nada, sólo muestra lo que haría.

runlevel

- Este comando nos informa del nivel de ejecución anterior y actual.
- Al ejecutarlo nos devuelve dos caracteres: El primer carácter es el modo de ejecución anterior. Cuando el carácter es N, significa que el sistema no ha cambiado de modo de ejecución desde el arranque. El segundo carácter indica el nivel de ejecución actual.

init

- Este comando permite cambiar el nivel de ejecución de un sistema en funcionamiento.
- Se ejecuta pasándole como parámetro el nivel de ejecución al que queremos cambiar.

Ejemplo: `init 6`

shutdown

- Este comando se utiliza para detener, apagar o reiniciar el sistema. Es el comando más adecuado para realizar estas acciones.
- El comando **shutdown** envía un mensaje estándar a todos los usuarios. También permite especificar cuando se apagará el sistema, para que los usuarios puedan terminar sus procesos.
- Sintaxis: `shutdown [opciones] [time] [Mensaje]`
Las opciones más utilizadas son:
 - r Reinicia o sistema.
 - P Apaga el sistema (es la opción por defecto)
 - H Detiene el sistema.
 - c Cancela un shutdown pendiente.
 - k No detiene, apaga o reinicia; sólo escribe el mensaje

- La forma más sencilla de ejecutarlo es pasándole el argumento temporal `now`. Ejecutaríamos `shutdown now`. Opcionalmente podemos indicarle un mensaje a enviar a todos los usuarios conectados.
- El argumento temporal también se puede especificar en el formato `hh:mm`, para un formato horario de 24 horas y `+m` (o sin el símbolo `+`) para especificar un período de `m` minutos sobre la hora actual.
Ejemplo: `shutdown -r +10 "Reinicio del sistema en 10 minutos"`
- También existen los comandos **halt**, **reboot** y **poweroff**.
 - **halt** es equivalente a: `shutdown -h now`
 - **reboot** es equivalente a: `shutdown -r now`

3.4. El proceso de inicialización Systemd

Systemd se ha creado para ofrecer un inicio más rápido y flexible que SysV.

El método de inicialización systemd cada vez es más utilizado en el mundo Linux. En la actualidad es el proceso de inicialización predeterminado que se emplea en distribuciones como Fedora, CentOS, Debian, Ubuntu.

Ventajas de systemd:

- Permite el arranque paralelo de procesos, usando sockets y reduciendo los tiempos de arranque
- Permite activar servicios bajo demanda o asociados a eventos por medio de D-Bus.
- Gestión de las dependencias entre servicios.
- Monitorización y logging de procesos en registros propios, filtrables
- Optimiza el uso de recursos utilizando cgroups
- Sintaxis de configuración más intuitiva
- Es compatible con scripts SysV

También tiene sus detractores, y recibió numerosas críticas por parte de miembros de la comunidad que consideran que systemd es una ruptura con la filosofía fundamental de Linux: "Haz algo simple y hazlo bien".

En sistemas systemd el programa `init` se substituye por systemd, que tiene el PID 1. Soportan las convenciones SysV para proporcionar compatibilidad, pero no utilizan el archivo `/etc/inittab`.

El concepto fundamental en systemd es el de **Unit** (unidad):

- Una **Unit** representa a un recurso administrable mediante systemd.

Con **systemd** se pueden gestionar muchos aspectos de configuración de un sistema: servicios, puntos de montaje, logs, estados del sistema, etc.

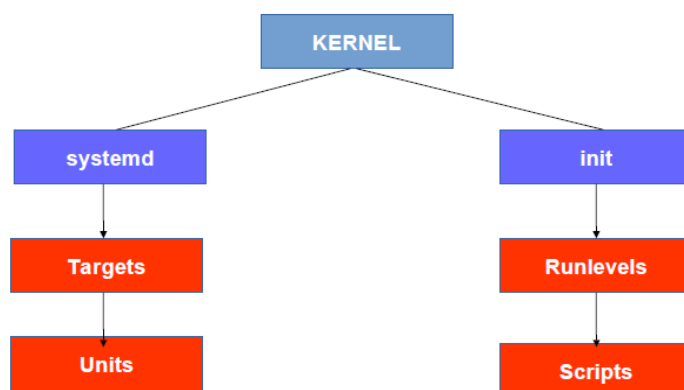
- La definición de las **Units** se realiza mediante los *Unit Files*:
 - Permiten la definición y administración de los aspectos de cada **Unit**.
 - Contienen directivas asociadas al tipo de **Unit**.
- Cada tipo de **Unit** está identificada por el sufijo de su *Unit File*.
Por ejemplo, para la gestión de servicios se utiliza el sufijo **.service**; para la gestión de dispositivos se utiliza el sufijo **.device**.
- Uno de los aspectos fundamentales gestionados mediante **units** son los servicios / demonios del sistema.

El proceso de inicialización **systemd** utiliza las **units** tipo **target** de la misma forma que SysV utiliza los niveles de ejecución, pero actúa ligeramente diferente. Cada **target** es nombrado en vez de numerado y está diseñado para llevar a cabo un propósito específico. Cada **target** representa un grupo diferente de servicios que debe ejecutarse en el sistema. Otra diferencia respecto a los niveles de ejecución de SysV es que en **systemd** un sistema puede tener activos varios targets al mismo tiempo. Existen dependencias entre **targets**, de forma que estos están interrelacionados.

La **unit** *default.target* equivale al concepto de nivel de ejecución por defecto en SysV. Es la **unit** que busca el programa **systemd** cuando se inicia. Suele establecerse como un enlace a un archivo **target** del directorio **/lib/systemd/system**.

Para facilitar la transición de SysV a **systemd**, existen **targets** que imitan los niveles de ejecución estándar 0-6 de SysV, con nombres entre **runlevel0.target** y **runlevel6.target**.

Systemd vs SysV





3.4.1. Gestión de Unit Files

Los Unit File tienen un nombre del tipo *nome_unit.tipo_unit*, por ejemplo *ssh.service* o *multi-user.target*.

Tipos de Units:

- **service**: gestión de servicios.
- **socket**: gestión de los sockets utilizados por otras **units** (como servicios).
- **device**: gestión de dispositivos (con udev)
- **mount** e **automount**: gestión de puntos de montaje.
- **swap**: gestión de espacios de intercambio.
- **target**: gestión de **targets**. Permiten agrupar varias **units** para poder iniciarlas al mismo tiempo.
- **path**: utilizada para la activación de *units.service* basada en eventos del sistema de archivos referenciado por el *path*.
- **timer**: programación de tareas (al estilo *cron* o *at*)
- **snapshot**: gestión de instantáneas del sistema.
- **slice**: gestión de recursos asociados a procesos usando *cgroup*.
- **scope**: gestión de procesos organizados y creados por *systemd*.

La sintaxis de definición de los *Unit Files* está basada en secciones. Cada sección indica un aspecto, en el que se define, a través de directivas, todos los parámetros asociados a ese aspecto. Algunas secciones de las *Unit Files* son:

- [**Unit**]: Define información sobre la **unit** y sus interrelaciones con otras .
 - Directivas: *After* / *Before* → Se ejecuta tras/antes la units listadas ;
Wants / *Require* → Establece dependencias poco/muy restrictivas;

- [**Install**]: Define directivas de gestión de **units**, estableciendo relaciones en aspectos como, por ejemplo, la relación de la **unit** con **targets** asociados.
- [**Service**]: Define directivas específicas de un servicio.
- [**Socket**]: Define directivas de configuración de sockets asociados a servicios.
- [**Mount**] e [**Automount**]: Define directivas para puntos de montaje.
- [**Swap**]: Directivas que definen y habilitan espacios de intercambio para páginas de memoria virtual anónimas.
- [**Timer**]: Definición y gestión de eventos temporales.

- [**Path**]: Monitorización del sistema de archivos.
- [**Slice**]: Gestión de asignación de recursos a los procesos.

El sistema `systemd` almacena los archivos de configuración de **Units** en el directorio `/lib/systemd/system` o `/etc/systemd/system`.

Los directorios `*.wants` (situados igualmente en ambos os directorios) contienen los enlaces simbólicos que apuntan a determinados servicios. Serán estos servicios los que se ejecuten con el **target** al que corresponde dicho directorio `wants`. Recordar que si un **target** precisa de otro, también serán cargados los servicios de este otro **target**.

Veamos un ejemplo de un archivo *Unit File*, en este caso el de `ssh.service`:

```
#cat
/lib/systemd/system/ssh.service

[Unit]
    Description=OpenBSD Secure
    Shell server
    After=network.target
    auditd.service

[Service]
    EnvironmentFile=/etc/default/
ssh
    ExecStart=/usr/sbin/sshd -D
    $SSHD_OPTS
    ExecReload=/bin/kill -HUP
    $MAINPID
    KillMode=process
    Restart=on-failure

[Install]
    WantedBy=multi-user.target
    Alias=sshd.service
#
```

Este archivo define el programa a ejecutar (`/usr/sbin/sshd`), junto con otras opciones, como los servicios a ejecutar antes de iniciar el servicio `sshd` (la línea `After`), en qué nivel de destino debe estar el sistema (la línea `WantedBy`) y cómo volver a cargar el programa (la línea `Restart`). Las `units` destino/target también usan archivos de configuración, que no definen programas sino las unidades de servicio a iniciar.

Como podemos observar, los archivos de configuración están formados por secciones identificadas por nombres entre corchetes, seguidas de asignaciones de valores a variables.

3.4.2. Gestión de Servicios systemd -> systemctl

Con el proceso de inicialización systemd se utiliza programa **systemctl**, para gestionar las Units. Podremos, entre otros, controlar los el servicios y los targets. Este programa acepta numerosas opciones y comandos. Por lo general se tendrá que pasar un nombre de Unit, que es el nombre de un servicio sobre el que actúa (u otro recurso). Si no se especifica el sufijo, systemctl asumirá que es *.service*.

Sintaxis de systemctl:

systemctl [opciones] comando [unit]

Los comandos más importantes de systemctl, relacionados con los servicios son:

- **list-units** → Muestra el estado actual de todas las units activadas. Este es el comando que toma por defecto si no se le pasa alguno..
- **start nombre** → Inicia la unit especificada.
- **stop nombre** → Detiene la unit especificada.
- **restart nombre** → Hace que la unit especificada se apague y se reinicie.
- **reload nombre** → Hace que la unit especificada vuelva a cargar su archivo de configuración.
- **status nombre** → Muestra el estado de la unit especificada (se puede pasar el PID en lugar de un nombre).
- **enable nombre** → Configura la unit para que se inicie en el siguiente arranque del ordenador.
- **disable nombre** → Configura la unit para que no se inicie en el siguiente arranque del ordenador.
- **is-enabled nombre** → Muestra si el servicio está configurado para iniciarse automáticamente al inicio.
- **list-unit-files** → Muestra todas las units disponibles.

Ejemplo → Detener el servicio sshd:

systemctl stop ssh.service

Los comandos más importantes de systemctl, relacionados con los targets son:

- **isolate name** → Inicia la unit target y detiene todas las demás
- **default** → Cambia a la unit target por defecto del sistema
- **get-default** → Muestra la unit target por defecto del sistema.
- **Set-default name** → Establece la unit target por defecto del sistema.

Ejemplos → Para cambiar la unidad en ejecución tendrá que usar el comando `isolate`. Así, para acceder a modo monousuario:

`systemctl isolate rescue.target`

Para regresar al destino predeterminado del sistema, utilice el comando `default`.

En `systemd` para reiniciar, apagar el sistema podemos utilizar:

- `systemctl poweroff` → Apaga el sistema
- `systemctl reboot` → Reinicia el sistema
- `systemctl suspend` → Suspende el sistema

3.4.3. `systemd` → `journalctl`

Una de las características más controvertidas del proceso de inicialización `systemd` es que no utiliza el sistema de registro `rsyslogd` estándar de Linux, sino que dispone de sus propios archivos de registro, que no se almacenan en formato de texto. El demonio *journald* recopila datos de todas las fuentes disponibles y los almacena en un formato binario para una manipulación fácil y dinámica.

Para ver los archivos de registro de `systemd` se necesitará usar el comando `journalctl`.

- **`journalctl -b`** → Muestra las entradas del diario desde el último reinicio
- **`journalctl -u unit`** → Muestra las entradas del diario generados por esa unit
- `journalctl -k` → Muestra sólo los mensajes del kernel

4. LINUX. Automatización de tareas

En este apartado veremos la utilización de comandos que permiten automatizar tareas repetitivas:

- `at`, `batch` permiten ejecutar tareas a una hora específica o bajo determinadas condiciones
- `cron` permite correr trabajos a intervalos regulares; programar tareas de `cron`

4.1. Ejecutar tareas a una determinada hora: `at`

- El comando `at` permite indicar el momento en que se quiere ejecutar un trabajo
- Sintaxis: **`at [opciones ] TIME`**



- Al ejecutar `at` pasamos a un nuevo prompt, que nos permite introducir comandos que se ejecutarán a la hora indicada:
 - para salvar el trabajo y salir CTRL-D
 - al terminar, la salida estándar se envía como un mail al usuario (si está instalado un agente de correo)
 - el trabajo se ejecuta mientras el sistema esté encendido a la hora indicada.

- Ejemplo:

```
$ at 11:45
warning: commands will be executed using /bin/sh
at> ls /tmp > lista
at> env DISPLAY=:0 zenity -info -text="Hola"
at> <EOT>
job 4 at Wed Nov 16 11:45:00 2005
```

- Algunas opciones:
 - `-f FILE` especifica un fichero conteniendo las acciones a realizar en vez de la entrada estándar
 - `-m` envía un mail al usuario cuando el trabajo se ha ejecutado, incluso aunque no haya salida.
 - `-M` nunca envía mail al usuario.
- TIME puede especificarse de varias formas:
 - HH:MM por ejemplo 12:54
 - HH:MM AM/PM, por ejemplo 1:35PM
 - HH:MM MMDDYY, por ejemplo 1:35PM 122505
 - now + numero unidades , donde unidades puede ser minutes, hours, days, o weeks:
\$ at now+2hours
 - today, tomorrow, por ejemplo 12:44tomorrow
 - midnight (00:00), noon (12:00), teatime (16:00)
- **Comandos relacionados:**
 - `atq` o `at -l` → lista los trabajos pendientes del usuario. Si es el superusuario, lista los trabajos de todos los usuarios
 - `atrm` o `at -d` o `at -r` → borra trabajos identificados por su número de trabajo.
 - `batch` ejecuta trabajos cuando la carga del sistema es baja:
 - el trabajo empieza en cuanto la carga caiga por debajo de 1.5



- la carga se obtiene del fichero /proc/loadavg
- atd → demonio encargado de ejecutar las órdenes

- **Ficheros de configuración:**

El administrador puede controlar la utilización de at mediante los ficheros /etc/at.allow y /etc/at.deny → Lista los usuarios que pueden y NO pueden usar at.

- Primero se chequea /etc/at.allow. Si está el usuario, puede usar at. Si no está o el fichero no existe, se chequea /etc/at.deny. Si el usuario no está en at.deny puede usar at.
- Si no existe ninguno de los dos ficheros, solo root puede ejecutar at.
- Para dar permiso para todos los usuarios crear sólo el fichero at.deny vacío

4.2. Ejecutar tareas periódicamente: cron.

- Para crear trabajos que se ejecuten periódicamente se utilizan el demonio cron.
- Cron responde a eventos temporales. En concreto , “se despierta” una vez por minuto, examina los archivos de configuración de los directorios /var/spool/cron y /etc/cron.d y el archivo /etc/crontab, ejecutando los comandos especificados en estos archivos de configuración si la hora coincide con la que figura en estos.
- Hay dos tipos de tareas cron: las del sistema y las de los usuarios. Las tareas cron del sistema se ejecutan como root y llevan a cabo tareas de mantenimiento por todo el sistema
- Cron está pensado para sistemas funcionando 24/7. Si el sistema está apagado a la hora de una acción cron, esa iteración no se realiza. Problema en sistemas de sobremesa y/o domésticos. Solución complementaria: Anacron

4.2.1. Tareas cron de usuario

- Para crear un cron de usuario se emplea el comando crontab (no confundir con fichero de configuración /etc/crontab) :
- La utilización de cron se gestiona a través de los ficheros /etc/cron.allow y /etc/cron.deny:
 - el comportamiento si no existen los ficheros depende de la configuración del sistema
 - en Debian, si no existen, todos los usuarios pueden usar crontab
- Los trabajos se especifican en un fichero crontab, que se guarda en /var/spool/cron/crontabs de la siguiente forma:

**minuto hora día_mes mes día_semana comando**

- Minuto → valor entre 0 y 59.
- Hora → valor entre 0 y 23.
- Día_mes → valor entre 1 y 31.
- Mes → valor entre 1 y 12.
- Día_semana de 0 a 7 (0 ó 7 domingo).

Además se pueden usar los caracteres:

- Asterisco(*) → indica cualquier valor
- Coma (,) → para indicar varios valores/lista de valores en un campo
- Guión (-) → para indicar un rango
- Barra oblicua (/) → para indicar cada cuanto/repeticiones
- Por ejemplo:
 - 1-5 para indicar de lunes a viernes en *Día_semana*
 - 15,45 para indicar que a "y-cuarto" y "menos-cuarto" en *Minutos*
 - */2 en el campo *hora*, indica realizar cada dos horas (0,2, 4, etc.)

(Este fichero puede tener también líneas de comentarios y de definición de variables)

- Ejemplos:
 - Borra el /tmp todos los días laborables a las 4:30 am:
30 4 * * 1-5 rm -rf /tmp/*
 - Escribe la hora, cada 15 minutos, durante la noche:
*/15 0-8,20-23 * * * echo Hora:\$(date)>/tmp/horas
- También se pueden utilizar las palabras especiales: @hourly, @daily, @mounthly ,@yearly, @reboot

Comando crontab: crear y editar los trabajos periódicos

- Sintaxis:
 - crontab [-u usuario] [-l |-e |-r]
 - crontab [-u usuario] fichero (instala un nuevo crontab desde un fichero)
- Opciones:
 - -u usuario crea o maneja el crontab de un usuario específico (sólo root)
 - -e edita el fichero crontab



- -l muestra el fichero crontab
- -r borra el fichero crontab
- Por ejemplo el usuario ejecutara el comando **crontab -e** para editar su archivo crontab y crear su tarea

4.2.2. Tareas cron del sistema

- El archivo /etc/crontab controla las tareas cron del sistema. Las líneas de este archivo tiene la siguiente forma:

minuto	hora	día_mes	mes	día_semana	nb_cuenta	comando
---------------	-------------	----------------	------------	-------------------	------------------	----------------

 - nb_cuenta es el nombre de la cuenta que se va a utilizar cuando se ejecute el programa
- El demonio cron busca ficheros en /var/spool/cron para ejecutarlos a la hora indicada:
 - Además también ejecuta las acciones indicadas en los ficheros /etc/crontab y en el directorio /etc/cron.d/
 - Estos ficheros suelen ser de mantenimiento del sistema
- El administrador puede crear scripts que se ejecuten con periodicidad horaria, diaria, semanal y mensual:
 - Sólo tiene que colocar esos scripts en los directorios /etc/cron.hourly,/etc/cron.daily, /etc/cron.weekly o /etc/cron.monthly.
 - La fecha y hora de ejecución de estos scripts se controla en el fichero /etc/crontab

4.3. Anacron

- Ejecuta asíncronamente tareas periódicas programadas:
 - Al iniciarse el sistema comprueba si hay tareas periódicas pendientes (que no se realizaron por estar el sistema apagado).De ser así, espera un cierto retardo y las ejecuta
- El archivo /etc/anacrontab controla las tareas anacron del sistema. Las líneas de este archivo tiene la siguiente forma:

período	retardo	id_del_trabajo	comando
----------------	----------------	-----------------------	----------------

 - El período se especifica en días (o como @monthly) y el retardo en minutos:



```
1      5   cron.daily   run-parts /etc/cron.daily
7      10  cron.weekly  run-parts /etc/cron.weekly
@monthly 15  cron.monthly run-parts /etc/cron.monthly
```

- En este ejemplo, cada día, los scripts en cron.daily se ejecutan 5 minutos después de iniciar el anacron, cada 7 días (10 minutos de espera), se ejecutan los scripts en cron.weekly y cada mes (15 minutos) los de cron.monthly.
- Limitaciones de anacron:
 - Solo para uso del administrador
 - Solo permite períodos de días (no vale para tareas que se deban ejecutar varias veces al día)